TIME SERIES FORECASTING - SPARKLING WINE DATASET

Problem Statement:

For this particular assignment, the data of different types of wine sales in the 20th century is to be analysed. Both of these data are from the same company but of different wines. As an analyst in the ABC Estate Wines, you are tasked to analyse and forecast Wine Sales in the 20th century.

--

We have given 2 datasets. Let's look at the first dataset, i.e., Sparkling.csv

In [1]:
```
#Importing all the important libraries and packages

import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
import statsmodels.api as sm

#importing filterwarnings
from warnings import filterwarnings
filterwarnings('ignore')
```

1. Read the data as an appropriate Time Series data and plot the data.

In [2]:
```
#Let's import the Sparkling.csv dataset

sparkling_data = pd.read_csv(r'D:\SHUBHANK !\GL\8. TOPIC 7 - Time Series Forecasting\Fi
```

In [3]:
```
#Now, let's check the head of the dataset

sparkling_data.head()
```

Out[3]:

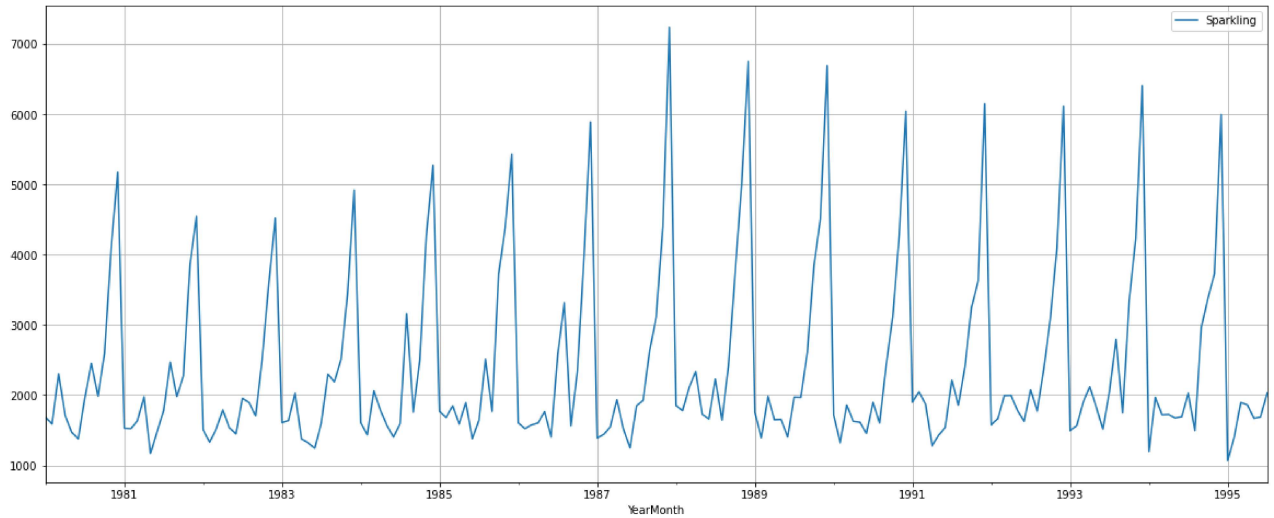
YearMonth	Sparkling
1980-01-01	1686
1980-02-01	1591
1980-03-01	2304
1980-04-01	1712
1980-05-01	1471

We can see that the Sparkling dataset has loaded perfectly.

In [4]:
```
#Now making the plot for the data
```

```
from pylab import rcParams

rcParams['figure.figsize'] = 20,8
```

In [5]:
```
sparkling_data.plot()

plt.grid()

plt.show()
```



By looking at the above plot, we can say that there is no trend in the data. The maximum sales happened between the year 1987 and 1989.

2. Perform appropriate Exploratory Data Analysis to understand the data and also perform decomposition.

In [6]:
```
#Let's look at the summary of the data

sparkling_data.describe()
```

Out[6]:

	Sparkling
count	187.000000
mean	2402.417112
std	1295.111540
min	1070.000000
25%	1605.000000
50%	1874.000000
75%	2549.000000
max	7242.000000

We can see that the maximum sales of the sparkling wine happened of 7242 and minimum sales of 1070.

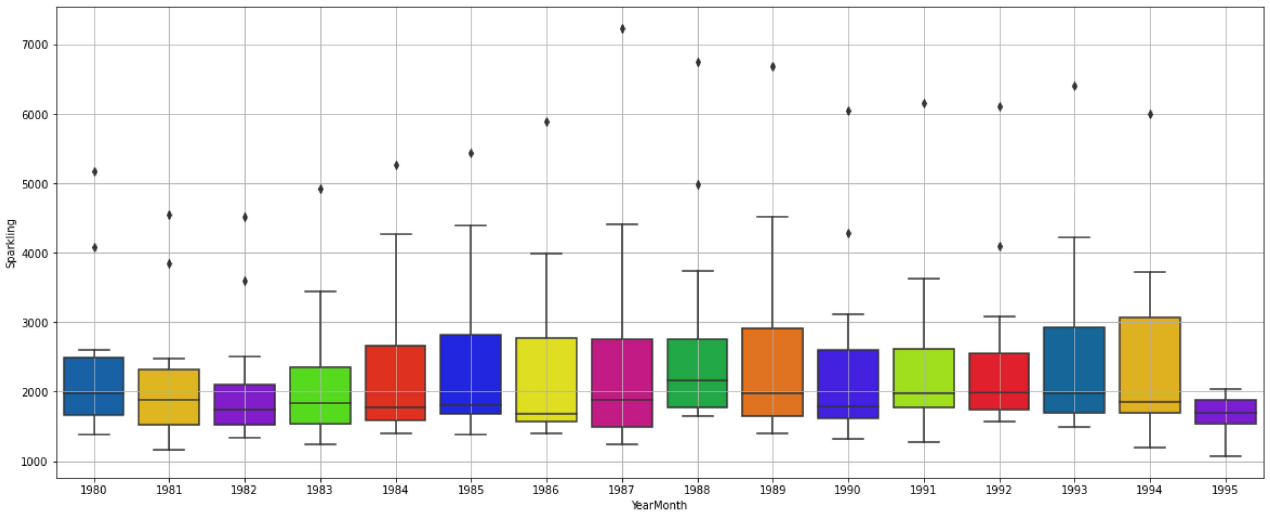
In [7]:
```
#Let's see the info

sparkling_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 187 entries, 1980-01-01 to 1995-07-01
Data columns (total 1 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Sparkling  187 non-null    int64
dtypes: int64(1)
memory usage: 2.9 KB
```

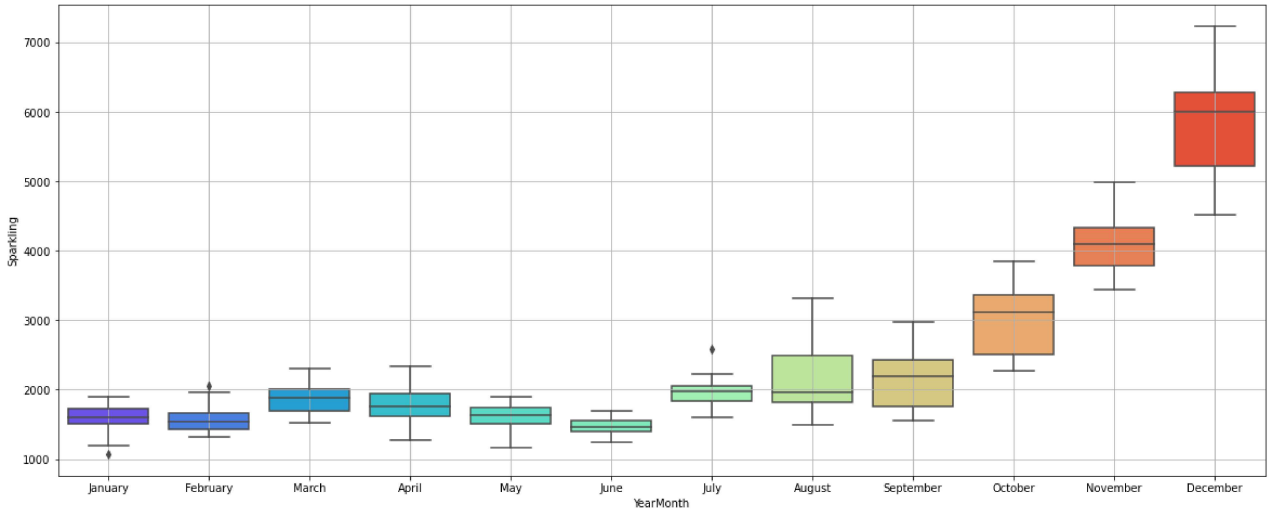
In [8]:
```
#Check the null values if any

sparkling_data.isna().sum()
```

Out[8]:
```
Sparkling    0
dtype: int64
```

In [9]:
```
#Boxplot to understand the sales of wine across years

#The below box plot is yearly basis

sns.boxplot(x=sparkling_data.index.year,y=sparkling_data['Sparkling'], palette='prism')
plt.grid()
plt.show()
```

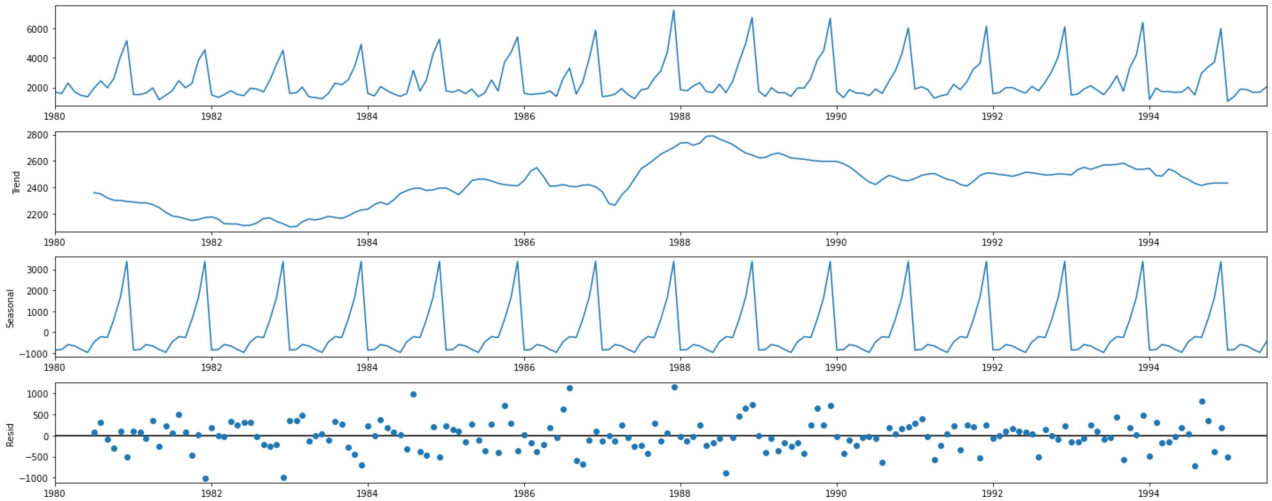


In [10]:
```
#The below box plot is monthly basis

sns.boxplot(x = sparkling_data.index.month_name(), y = sparkling_data['Sparkling'], pal
plt.grid()
plt.show()
```

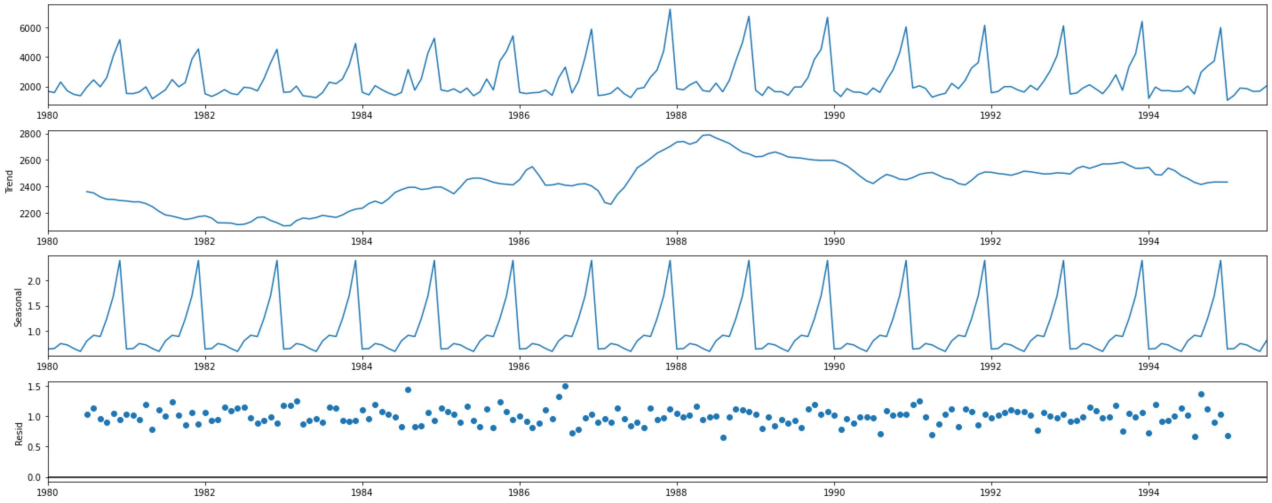


We can see that from January till June, the sales are not fluctuating too much but starting from July till December, we can see that sales increased. The maximum number of sales happened in the month of December and the lowest number of sales observed in the month of June.

In [11]:
```
#Decompose the data into additive model

from statsmodels.tsa.seasonal import seasonal_decompose

add_decomposition = seasonal_decompose(sparkling_data,model='additive')

add_decomposition.plot();
```



In [12]:
```
#Decompose the data into multiplicative model

from statsmodels.tsa.seasonal import seasonal_decompose

mul_decomposition = seasonal_decompose(sparkling_data,model='multiplicative')

mul_decomposition.plot();
```



After observing the additive and multiplicative models, we can see that the trend and seasonality of both the models are same, and also we can see the patterns in residual also for both the models. So in such case, it is better to go with additive model.

For our further analysis, we will use additive model.

3.Split the data into training and test. The test data should start in 1991.

In [13]:
```
#As given in the question, that we have to split the data into train and test set but t

#Let's first check the shape of the data

sparkling_data.shape
```

Out[13]:
```
(187, 1)
```

In [14]:
```
#As given in the question, that we have to split the data into train and test set but t

#Now, let's split the dataset

train = sparkling_data[sparkling_data.index.year < 1991]
test = sparkling_data[sparkling_data.index.year >= 1991]
```

In [15]:
```
#Now, let see the shape individually

print(train.shape)
print(test.shape)
```

```
(132, 1)
(55, 1)
```

In [16]:
```
train.tail()
```

Out[16]:

YearMonth	Sparkling

YearMonth	Sparkling
1990-08-01	1605
1990-09-01	2424
1990-10-01	3116
1990-11-01	4286
1990-12-01	6047

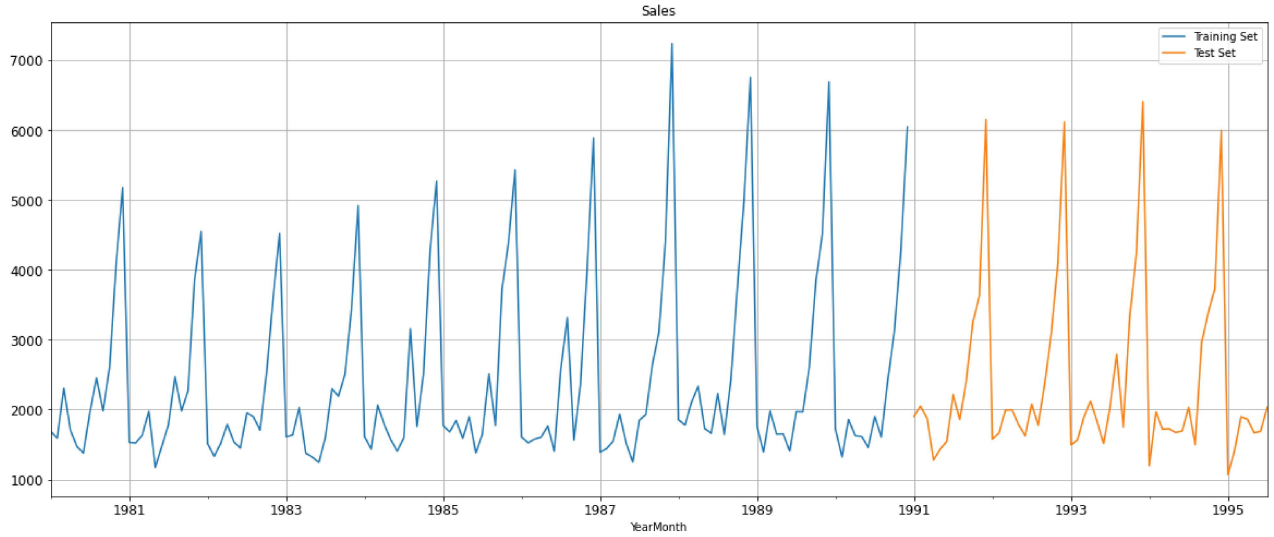
In [17]:
```
test.head()
```

Out[17]:

YearMonth	Sparkling
1991-01-01	1902
1991-02-01	2049
1991-03-01	1874
1991-04-01	1279
1991-05-01	1432

We can see that the last few rows of the train set is upto the 12th month of the year 1990 and from 1st month of the year 1991, the test set starts.

In [18]:
```
#Now let's plot the train and test set

train['Sparkling'].plot(fontsize = 12)
test['Sparkling'].plot(fontsize = 12)

plt.grid()
plt.legend(['Training Set','Test Set'])
plt.title('Sales')
plt.show()
```



Our data is split into train and test with 137 records in train and remaining 55 records in test. We can also look at the plot above.

4. Build various exponential smoothing models on the training data and evaluate the model using RMSE on the test data. Other models such as regression,naïve forecast models, simple average models etc. should also be built on the training data and check the performance on the test data using RMSE.

Building Linear Regression Model

In [19]:
```
#first import the LinearRegression package from sklearn library

from sklearn.linear_model import LinearRegression
```

In [20]:
```
#For Linear Regression model for our time series data, first we need to provide the tim

train_time = [i+1 for i in range(len(train))]

print('Training Time instance: \n\n', train_time)
```

```
Training Time instance:

 [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46,
47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 11
0, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127,
128, 129, 130, 131, 132]
```

In [21]:
```
test_time = [i+133 for i in range(len(test))]

print('Test Time instance: \n\n', test_time)
```

```
Test Time instance:

 [133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 1
```

```
50, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167,
168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 18
5, 186, 187]
```

In [22]:
```
#Copying the train and test to train_lr and test_lr for our further analysis

train_lr = train.copy()
test_lr = test.copy()
```

In [23]:
```
#Now, adding one more column for time instance

train_lr['Time_instance'] = train_time

test_lr['Time_instance'] = test_time
```

In [24]:
```
#Time_instance column has been added for both sets, now checking the head of both set

train_lr.head()
```

Out[24]:

YearMonth	Sparkling	Time_instance
1980-01-01	1686	1
1980-02-01	1591	2
1980-03-01	2304	3
1980-04-01	1712	4
1980-05-01	1471	5

In [25]:
```
test_lr.head()
```

Out[25]:

YearMonth	Sparkling	Time_instance
1991-01-01	1902	133
1991-02-01	2049	134
1991-03-01	1874	135
1991-04-01	1279	136
1991-05-01	1432	137

In [26]:
```
#Now further we will fit the sparkling and time_instance columns of the train set into

LR = LinearRegression()

LR.fit(train_lr[['Time_instance']], train_lr['Sparkling'].values)
```

```
LinearRegression()
```

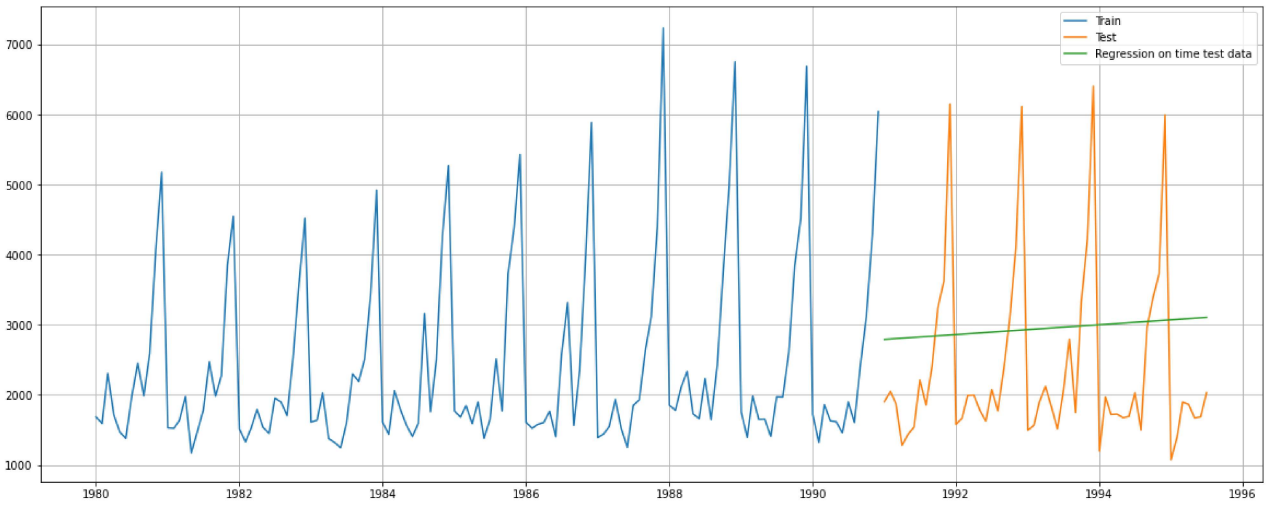
Out[26]:

In [27]:
```
#Now, checking the predictions on test set

LR_test_predict = LR.predict(test_lr[['Time_instance']])

#Now, adding one more column for 'Regression_on_time'  in test

test_lr['Regression_on_time'] = LR_test_predict

test_lr.head()
```

Out[27]:

YearMonth	Sparkling	Time_instance	Regression_on_time
1991-01-01	1902	133	2791.652093
1991-02-01	2049	134	2797.484752
1991-03-01	1874	135	2803.317410
1991-04-01	1279	136	2809.150069
1991-05-01	1432	137	2814.982727

In [28]:
```
#Plotting

plt.plot(train['Sparkling'],label = 'Train')
plt.plot(test['Sparkling'], label = 'Test')
plt.plot(test_lr['Regression_on_time'], label = 'Regression on time test data')

plt.legend(loc = 'best')
plt.grid()
```



In [29]:
```
#Now we'll check the Root Mean Square Error (RMSE) value on Test Data:

#Importing metrics package from sklern library:

from sklearn import metrics
```

```
RMSE_LR_test = metrics.mean_squared_error(test['Sparkling'],LR_test_predict ,squared=Fa

print("RMSE value for Regression_on_time forecast on the Test Data is %3.2f" %(RMSE_LR_
```

```
RMSE value for Regression_on_time forecast on the Test Data is 1389.14
```

In [30]:
```
#Comparing and Showing the results in dataframe


resultsDf = pd.DataFrame({'Test RMSE': [RMSE_LR_test]},index=['Regression_on_time'])

resultsDf
```

Out[30]:

	Test RMSE
Regression_on_time	1389.135175

We have calculated the Root Mean Square Error on the test data for the Linear Regression Model. Now we will check RMSE for the other models and compare all the models to finalize the best model.

Building Naive Model

In Naive Model, we know that the prediction for tomorrow is the same as today and the prediction for day after tomorrow is tomorrow and since the prediction of tomorrow is same as today, therefore the prediction for day after tomorrow is also today. So, by using this approach we will further calculate the RMSE value.

In [31]:
```
#Copying from the train and test set for further analysis:

train_nm = train.copy()
test_nm = test.copy()
```

In [32]:
```
train_nm.head()
```

Out[32]:

	Sparkling
YearMonth	
1980-01-01	1686
1980-02-01	1591
1980-03-01	2304
1980-04-01	1712
1980-05-01	1471

In [33]:
```
train_nm.tail()
```

Out[33]:

	Sparkling
YearMonth	

YearMonth	Sparkling
1990-08-01	1605
1990-09-01	2424
1990-10-01	3116
1990-11-01	4286
1990-12-01	6047

train set holds the data upto year 1990.

In [34]:
```
test_nm.head()
```

Out[34]:

YearMonth	Sparkling
1991-01-01	1902
1991-02-01	2049
1991-03-01	1874
1991-04-01	1279
1991-05-01	1432

In [35]:
```
test_nm.tail()
```

Out[35]:

YearMonth	Sparkling
1995-03-01	1897
1995-04-01	1862
1995-05-01	1670
1995-06-01	1688
1995-07-01	2031

In [36]:
```
#now we will create the new column in the test_nm set

test_nm['naive'] = np.asarray(train['Sparkling'])[len(np.asarray(train['Sparkling']))-1
```

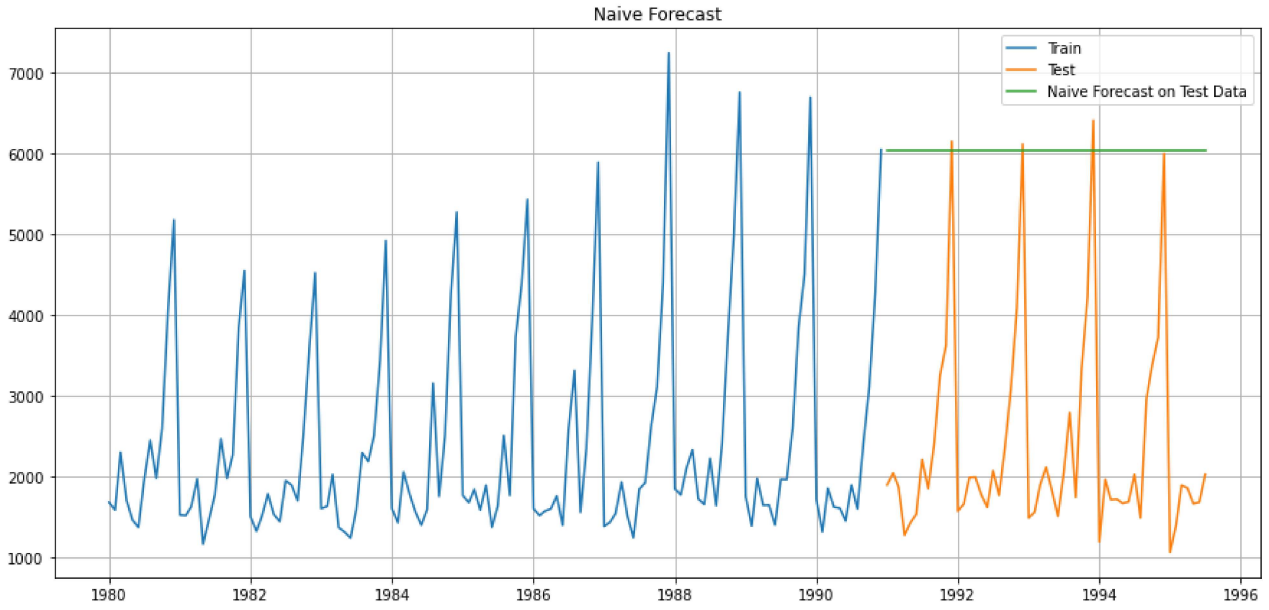
In [37]:
```
#now checking the head of the

test_nm['naive'].head()
```

Out[37]:
```
YearMonth
```

```
1991-01-01    6047
1991-02-01    6047
1991-03-01    6047
1991-04-01    6047
1991-05-01    6047
Name: naive, dtype: int64
```

In [38]:

```
#Plotting

plt.figure(figsize=(15,7))

plt.plot(train_nm['Sparkling'], label='Train')
plt.plot(test['Sparkling'], label='Test')
plt.plot(test_nm['naive'], label='Naive Forecast on Test Data')
plt.legend(loc='best')
plt.title("Naive Forecast")
plt.grid();
```



In [39]:

```
#Evaluation

#Calculating the Root Mean Square Error on Test Data


RMSE_NM_test = metrics.mean_squared_error(test['Sparkling'],test_nm['naive'],squared=Fa

print('RMSE value for Naive Model on testing set is %3.3f' %(RMSE_NM_test))
```

```
RMSE value for Naive Model on testing set is 3864.279
```

In [40]:

```
#Comparing and Showing the results in dataframe

resultsDf_2 = pd.DataFrame({'Test RMSE': [RMSE_NM_test]},index=['NaiveModel'])

resultsDf = pd.concat([resultsDf, resultsDf_2])

resultsDf
```

Out[40]:

	Test RMSE

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352

After calculating the RMSE on test set for Naive model, we can see the clear big difference in the values, that Naive model is giving very high error rate than the Linear Regression model. So at this point of time we can say that out of these two models, the Linear Regression is good. But before concluding and finalizing the end result, we need to look for the rmse values of other models as well, like Simple Average, Moving Average, Simple Exponential etc.

Building Simple Average Model

In [41]:
```
#Copying the train and test set

train_SA = train.copy()
test_SA = test.copy()
```

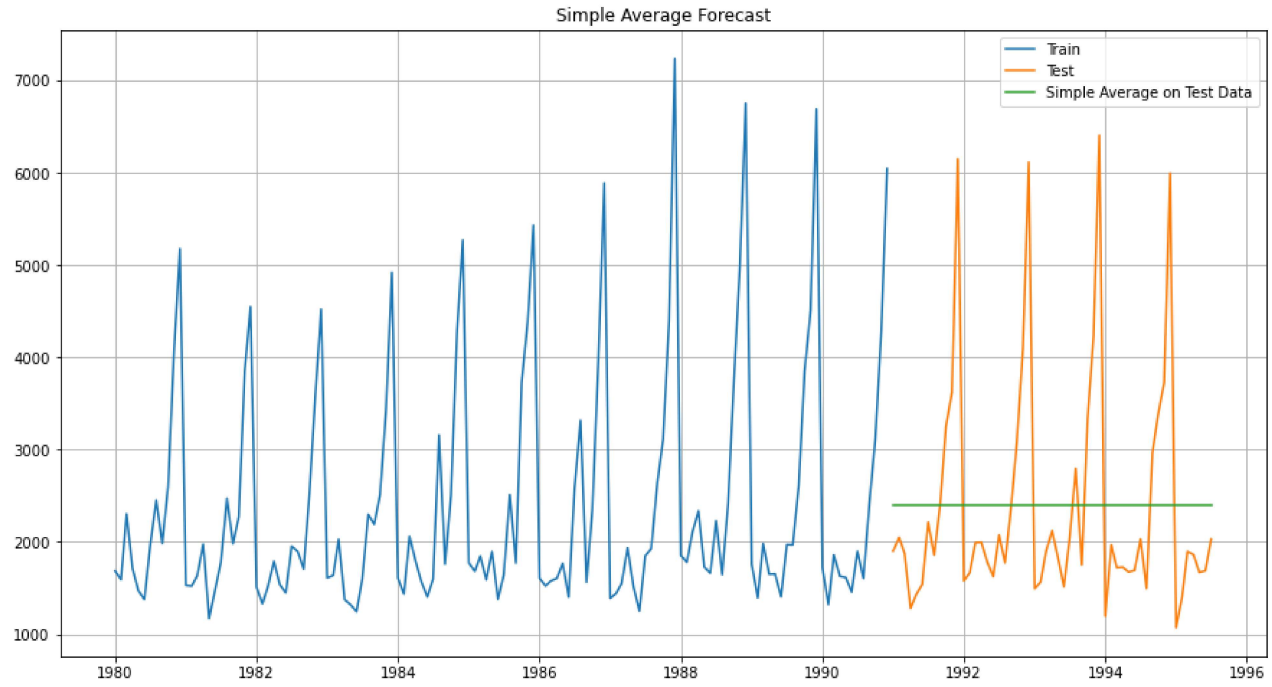
In [42]:
```
#In Simple Average model we will take the average of the training values and then will

test_SA['mean_forecast'] = train['Sparkling'].mean()
test_SA.head()
```

Out[42]:

YearMonth	Sparkling	mean_forecast
1991-01-01	1902	2403.780303
1991-02-01	2049	2403.780303
1991-03-01	1874	2403.780303
1991-04-01	1279	2403.780303
1991-05-01	1432	2403.780303

Mean forecast coming out for the training values is 2403.78. Now, let's plot it and have a look.

In [43]:
```
#Plotting

plt.figure(figsize=(15,8))
plt.plot(train_SA['Sparkling'], label='Train')
plt.plot(test_SA['Sparkling'], label='Test')
plt.plot(test_SA['mean_forecast'], label='Simple Average on Test Data')
plt.legend(loc='best')
plt.title("Simple Average Forecast")
plt.grid();
```



We can see the green line representing the mean value and we know that it is at 2403.78. Now let's evaluate the model.

In [44]:
```
#RMSE for the test data:

RMSE_SA_test = metrics.mean_squared_error(test['Sparkling'], test_SA['mean_forecast'],s

print("RMSE value for Simple Average Forecast on Test data is %3.3f" %(RMSE_SA_test))
```

```
RMSE value for Simple Average Forecast on Test data is 1275.082
```

In [45]:
```
#Now again compare it with other previous models

resultsDf_3 = pd.DataFrame({'Test RMSE': [RMSE_SA_test]}, index=['Simple_Average_Model'
resultsDf = pd.concat([resultsDf, resultsDf_3])
resultsDf
```

Out[45]:

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804

So now again, there we can see the major change in RMSE value, the error for the simple average model is coming down gradually and here it seems to be the winner from above all models. But still there are other models for which we need to calculate the RMSE values. Simple Average Model has the lowest RMSE value, which is 1275.08

Building Moving Average Model

In [46]:
```
#Here we need to take the entire data to calculate the moving average model
```

```
#Copying the original entire sparkling_data dataset

mov_avg = sparkling_data.copy()
```

In [47]:
```
mov_avg.head()
```

Out[47]:

YearMonth	Sparkling
1980-01-01	1686
1980-02-01	1591
1980-03-01	2304
1980-04-01	1712
1980-05-01	1471

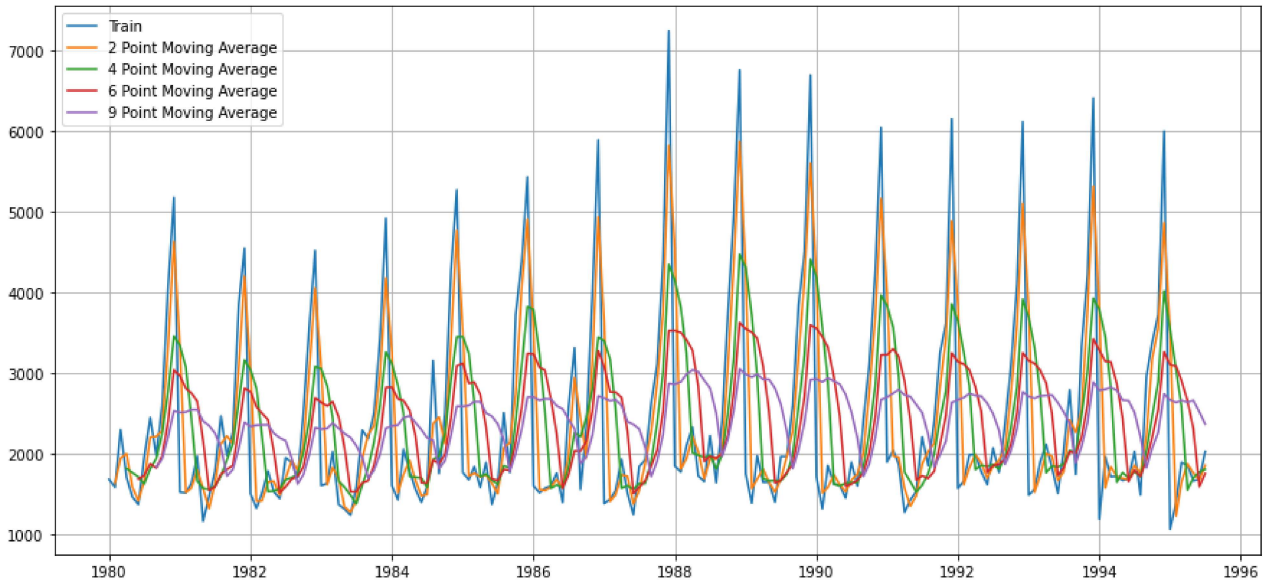
For this model, we need to calculate moving averages for different intervals. The best interval can be determined by the maximum accuracy with the minimum error.

In [48]:
```
mov_avg['Trailing_2'] = mov_avg['Sparkling'].rolling(2).mean()
mov_avg['Trailing_4'] = mov_avg['Sparkling'].rolling(4).mean()
mov_avg['Trailing_6'] = mov_avg['Sparkling'].rolling(6).mean()
mov_avg['Trailing_9'] = mov_avg['Sparkling'].rolling(9).mean()
mov_avg.head()
```

Out[48]:

YearMonth	Sparkling	Trailing_2	Trailing_4	Trailing_6	Trailing_9
1980-01-01	1686	NaN	NaN	NaN	NaN
1980-02-01	1591	1638.5	NaN	NaN	NaN
1980-03-01	2304	1947.5	NaN	NaN	NaN
1980-04-01	1712	2008.0	1823.25	NaN	NaN
1980-05-01	1471	1591.5	1769.50	NaN	NaN

In [49]:
```
#Plotting on the whole data

plt.figure(figsize=(15,7))
plt.plot(mov_avg['Sparkling'], label='Train')
plt.plot(mov_avg['Trailing_2'], label='2 Point Moving Average')
plt.plot(mov_avg['Trailing_4'], label='4 Point Moving Average')
plt.plot(mov_avg['Trailing_6'],label = '6 Point Moving Average')
plt.plot(mov_avg['Trailing_9'],label = '9 Point Moving Average')
plt.legend(loc = 'best')
plt.grid();
```



In [50]:
```
#Creating train and test set

trailing_MA_train = mov_avg[0:int(len(mov_avg)*0.709677)]
trailing_MA_test = mov_avg[int(len(mov_avg)*0.709677):]
```

In [51]:
```
#Now, let's check the count for trailing 2, trailing 4, trailing 6 and trailing 9 movin

trailing_MA_train.count()
```

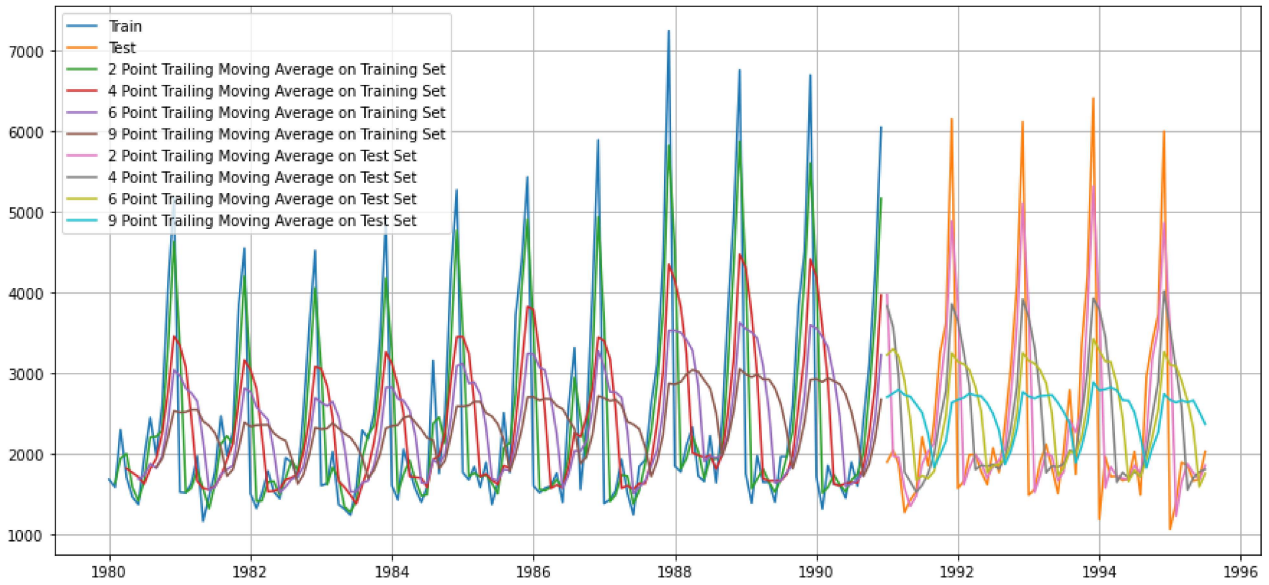
Out[51]:
```
Sparkling     132
Trailing_2    131
Trailing_4    129
Trailing_6    127
Trailing_9    124
dtype: int64
```

In [52]:
```
#Now, let's check the count for trailing 2, trailing 4, trailing 6 and trailing 9 movin

trailing_MA_test.count()
```

Out[52]:
```
Sparkling     55
Trailing_2    55
Trailing_4    55
Trailing_6    55
Trailing_9    55
dtype: int64
```

In [53]:
```
#Plotting

plt.figure(figsize=(15,7))
plt.plot(trailing_MA_train['Sparkling'], label='Train')
plt.plot(trailing_MA_test['Sparkling'], label='Test')

plt.plot(trailing_MA_train['Trailing_2'],label='2 Point Trailing Moving Average on Trai
plt.plot(trailing_MA_train['Trailing_4'],label='4 Point Trailing Moving Average on Trai
plt.plot(trailing_MA_train['Trailing_6'],label= '6 Point Trailing Moving Average on Tra
plt.plot(trailing_MA_train['Trailing_9'],label= '9 Point Trailing Moving Average on Tra
```

```
plt.plot(trailing_MA_test['Trailing_2'],label='2 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_4'],label='4 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_6'],label= '6 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_9'],label= '9 Point Trailing Moving Average on Test
plt.legend(loc = 'best')
plt.grid();
```



In [54]:

```
#Evaluation on test data for RMSE values of trailing 2,4,6 and 9

#RMSE for Test Data --> 2 point Trailing MA

RMSE_MA_test_2 = metrics.mean_squared_error(test['Sparkling'],trailing_MA_test['Trailin
print("For 2 point Moving Average Model forecast on the Training Data,  RMSE is %3.3f"

#RMSE for Test Data --> 4 point Trailing MA

RMSE_MA_test_4 = metrics.mean_squared_error(test['Sparkling'],trailing_MA_test['Trailin
print("For 4 point Moving Average Model forecast on the Training Data,  RMSE is %3.3f"

#RMSE for Test Data --> 6 point Trailing MA

RMSE_MA_test_6 = metrics.mean_squared_error(test['Sparkling'],trailing_MA_test['Trailin
print("For 6 point Moving Average Model forecast on the Training Data,  RMSE is %3.3f"

#RMSE for Test Data --> 9 point Trailing MA

RMSE_MA_test_9 = metrics.mean_squared_error(test['Sparkling'],trailing_MA_test['Trailin
print("For 9 point Moving Average Model forecast on the Training Data,  RMSE is %3.3f "
```

```
For 2 point Moving Average Model forecast on the Training Data,  RMSE is 813.401
For 4 point Moving Average Model forecast on the Training Data,  RMSE is 1156.590
For 6 point Moving Average Model forecast on the Training Data,  RMSE is 1283.927
For 9 point Moving Average Model forecast on the Training Data,  RMSE is 1346.278
```

We can clearly see that the RMSE for the trailing moving average model forecast for 2 point is best among all. The best RMSE is 813.401

In [55]:

```
#Now let's compare all the rmse for moving average models with the other previous model
```

```
resultsDf_4 = pd.DataFrame({'Test RMSE': [RMSE_MA_test_2, RMSE_MA_test_4 ,RMSE_MA_test_
                           ,index=['2pointTrailingMovingAverage','4pointTrailingMovingA
                                   ,'6pointTrailingMovingAverage','9pointTrailingMoving
```

In [56]:
```
resultsDf = pd.concat([resultsDf, resultsDf_4])

resultsDf
```

Out[56]:

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315

There is a clear view that 2 point trailing moving average is best model to go with for further analysis. But we have not yet build the simple exponential smoothing model, so before that we cannot finalize anything.

Simple Exponential Smoothing model

Simple Exponential Smoothing, is a time series forecasting method for univariate data without a trend or seasonality. It requires a single parameter, called alpha (a), also called the smoothing factor or smoothing coefficient.

This method is suitable for forecasting data with no clear trend or seasonal pattern.

SES is represented as: $F_{t+1} = \alpha Y_t + (1-\alpha) F_t$

Parameter α is called the smoothing constant and its value lies between 0 and 1. Since the model uses only one smoothing constant, it is called Single Exponential Smoothing.

In [57]:
```
#importing ExponentialSmoothing, SimpleExpSmoothing and Holt

from statsmodels.tsa.api import ExponentialSmoothing, SimpleExpSmoothing, Holt
```

In [58]:
```
#copying the train and test set

train_SES = train.copy()
test_SES = test.copy()
```

SimpleExpSmoothing class must be instantiated and passed the training data.

The fit() function is then called providing the fit configuration, the alpha value, smoothing_level. If this is omitted or set to None, the model will automatically optimize the value.

In [59]:
```
#creating class

model_SES = SimpleExpSmoothing(train_SES['Sparkling'], initialization_method='estimated
```

In [60]:
```
# Fitting the Simple Exponential Smoothing model and asking python to choose the optima
model_SES_autofit = model_SES.fit(optimized=True)
```

In [61]:
```
#checking the autofit parameters

model_SES_autofit.params
```

Out[61]:
```
{'smoothing_level': 0.07029120765764557,
 'smoothing_trend': nan,
 'smoothing_seasonal': nan,
 'damping_trend': nan,
 'initial_level': 1764.0137060346985,
 'initial_trend': nan,
 'initial_seasons': array([], dtype=float64),
 'use_boxcox': False,
 'lamda': None,
 'remove_bias': False}
```

In [62]:
```
# Using the fitted model on the training set to forecast on the test set

#making a separate column name 'predict'

test_SES['predict'] = model_SES_autofit.forecast(steps=len(test_SES))

display(test_SES)
```

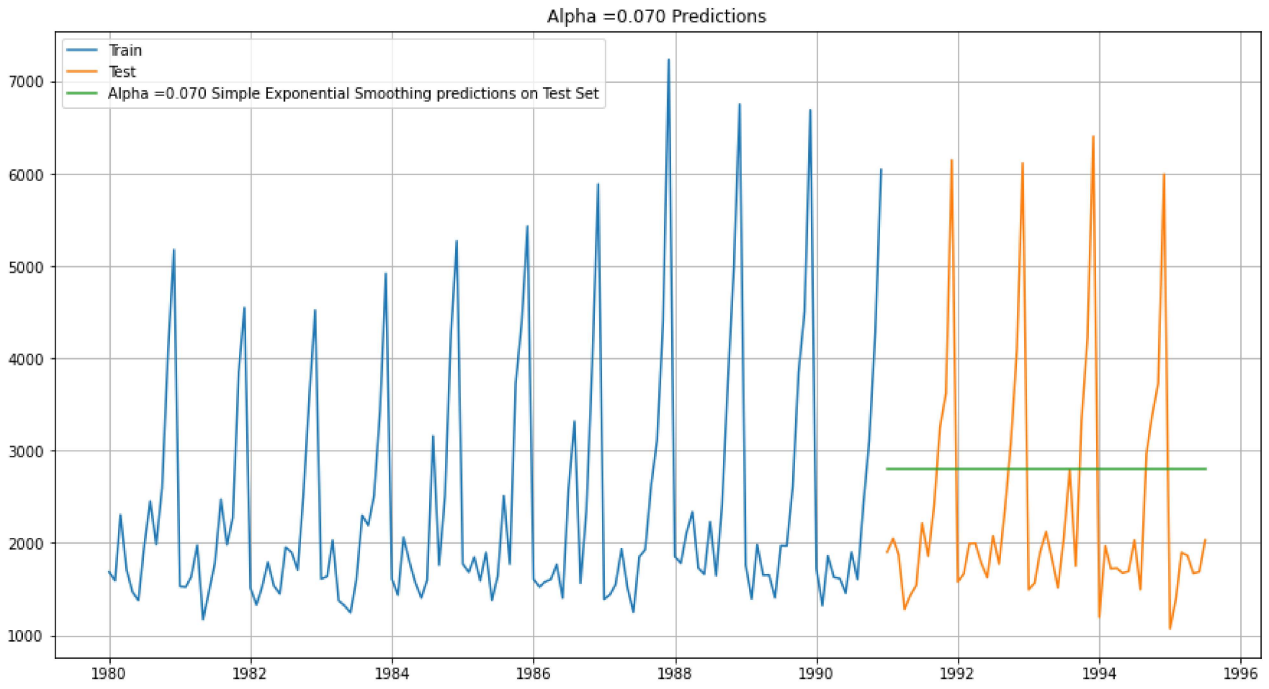
YearMonth	Sparkling	predict
1991-01-01	1902	2804.675124
1991-02-01	2049	2804.675124
1991-03-01	1874	2804.675124
1991-04-01	1279	2804.675124
1991-05-01	1432	2804.675124
1991-06-01	1540	2804.675124
1991-07-01	2214	2804.675124
1991-08-01	1857	2804.675124
1991-09-01	2408	2804.675124
1991-10-01	3252	2804.675124
1991-11-01	3627	2804.675124
1991-12-01	6153	2804.675124

YearMonth	Sparkling	predict
1992-01-01	1577	2804.675124
1992-02-01	1667	2804.675124
1992-03-01	1993	2804.675124
1992-04-01	1997	2804.675124
1992-05-01	1783	2804.675124
1992-06-01	1625	2804.675124
1992-07-01	2076	2804.675124
1992-08-01	1773	2804.675124
1992-09-01	2377	2804.675124
1992-10-01	3088	2804.675124
1992-11-01	4096	2804.675124
1992-12-01	6119	2804.675124
1993-01-01	1494	2804.675124
1993-02-01	1564	2804.675124
1993-03-01	1898	2804.675124
1993-04-01	2121	2804.675124
1993-05-01	1831	2804.675124
1993-06-01	1515	2804.675124
1993-07-01	2048	2804.675124
1993-08-01	2795	2804.675124
1993-09-01	1749	2804.675124
1993-10-01	3339	2804.675124
1993-11-01	4227	2804.675124
1993-12-01	6410	2804.675124
1994-01-01	1197	2804.675124
1994-02-01	1968	2804.675124
1994-03-01	1720	2804.675124
1994-04-01	1725	2804.675124
1994-05-01	1674	2804.675124
1994-06-01	1693	2804.675124
1994-07-01	2031	2804.675124
1994-08-01	1495	2804.675124

YearMonth	Sparkling	predict
1994-09-01	2968	2804.675124
1994-10-01	3385	2804.675124
1994-11-01	3729	2804.675124
1994-12-01	5999	2804.675124
1995-01-01	1070	2804.675124
1995-02-01	1402	2804.675124
1995-03-01	1897	2804.675124
1995-04-01	1862	2804.675124
1995-05-01	1670	2804.675124
1995-06-01	1688	2804.675124
1995-07-01	2031	2804.675124

In [63]:

```
## Plotting

plt.figure(figsize=(15,8))
plt.plot(train_SES['Sparkling'], label='Train')
plt.plot(test_SES['Sparkling'], label='Test')

plt.plot(test_SES['predict'], label='Alpha =0.070 Simple Exponential Smoothing predicti

plt.legend(loc='best')
plt.grid()
plt.title('Alpha =0.070 Predictions');
```



In [64]:

```
#Model Evaluation for aplha = 0.070 Simple Exponential Smoothing

#RMSE for Test Data


RMSE_SES_test = metrics.mean_squared_error(test_SES['Sparkling'],test_SES['predict'],sq

print('RMSE value for alpha value 0.070 Simple Exponential Smoothing Model forecast on
```

```
RMSE value for alpha value 0.070 Simple Exponential Smoothing Model forecast on the Test
Data is 1338.008
```

In [65]:
```
#Comparing again

resultsDf_5 = pd.DataFrame({'Test RMSE': [RMSE_SES_test]},index=['SimpleExponentialSmoo
resultsDf = pd.concat([resultsDf, resultsDf_5])
resultsDf
```

Out[65]:

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384

Triple Exponential Smoothing or Holt - Winter's Model:

In [66]:
```
#Copying

TES_train = train.copy()
TES_test = test.copy()
```

In [67]:
```
model_TES = ExponentialSmoothing(TES_train['Sparkling'],trend='additive',seasonal='mult
```

In [68]:
```
model_TES_autofit = model_TES.fit()
```

In [69]:
```
model_TES_autofit.params
```

Out[69]:
```
{'smoothing_level': 0.111108139467838,
 'smoothing_trend': 0.06172875597197263,
 'smoothing_seasonal': 0.3950479631147446,
 'damping_trend': nan,
 'initial_level': 1639.9340657558994,
 'initial_trend': -12.22494561218149,
```

```
'initial_seasons': array([1.06402008, 1.02352078, 1.40671876, 1.20165543, 0.97593   ,
       0.97100155, 1.31897446, 1.69588922, 1.3895294 , 1.81476396,
       2.85150039, 3.62470528]),
'use_boxcox': False,
'lamda': None,
'remove_bias': False}
```

In [70]:

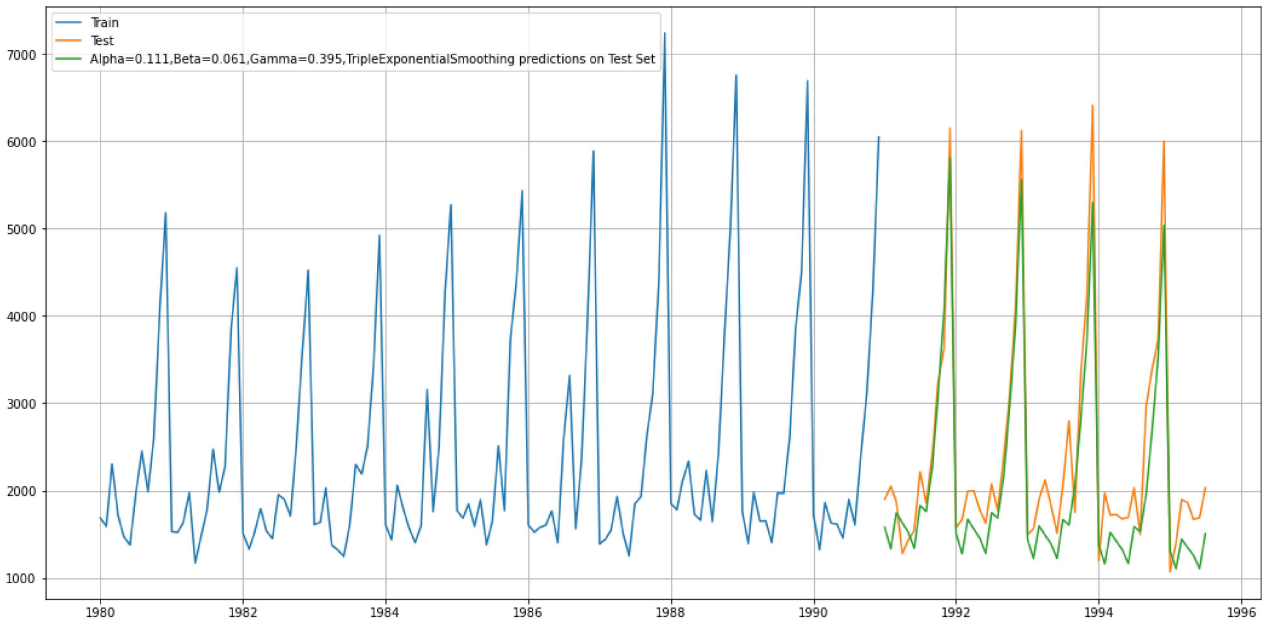
```
TES_test['auto_predict'] = model_TES_autofit.forecast(steps=len(test))
TES_test.head()
```

Out[70]:

YearMonth	Sparkling	auto_predict
1991-01-01	1902	1577.224489
1991-02-01	2049	1333.677558
1991-03-01	1874	1745.945679
1991-04-01	1279	1630.411925
1991-05-01	1432	1523.289070

In [71]:

```
#Plotting

plt.figure(figsize=(18,9))
plt.plot(TES_train['Sparkling'], label='Train')
plt.plot(TES_test['Sparkling'], label='Test')
plt.plot(TES_test['auto_predict'], label='Alpha=0.111,Beta=0.061,Gamma=0.395,TripleExpo
plt.legend(loc='best')
plt.grid();
```



In [72]:

```
#RMSE on test data

#We got the alpha , beta and gamma value from the autofit parameters

RMSE_TES_test = metrics.mean_squared_error(TES_test['Sparkling'],TES_test['auto_predict
```

```
print("RMSE for the Triple Exponential Smoothing Model forecast for the given values on
```

```
RMSE for the Triple Exponential Smoothing Model forecast for the given values on the Tes
t Data is 469.768
```

In [73]:

```
#Comparing

resultsDf_7 = pd.DataFrame({'Test RMSE': [RMSE_TES_test]} ,index=['TripleExponentialSmo

resultsDf = pd.concat([resultsDf, resultsDf_7])

resultsDf
```

Out[73]:

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970

We found out the lowest RMSE value on test data by using Triple Exponential Smoothing with the values of alpha (0.111) , value of beta (0.061) and with the value of gamma (0.395). So now we can see till now the best model amongst all is Triple Exponential Smoothing model.

In [74]:

```
#Now let's print all the values in sorted order

resultsDf.sort_values(by=['Test RMSE'])
```

Out[74]:

	Test RMSE
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
Simple_Average_Model	1275.081804
6pointTrailingMovingAverage	1283.927428
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
9pointTrailingMovingAverage	1346.278315
Regression_on_time	1389.135175

	Test RMSE
NaiveModel	3864.279352

From the above sorted dataframe we can see that the lowest rmse value is 469.76 which we got by using triple exponential smoothing model.

5. Check for the stationarity of the data on which the model is being built on using appropriate statistical tests and also mention the hypothesis for the statistical test. If the data is found to be non-stationary, take appropriate steps to make it stationary. Check the new data for stationarity and comment. Note: Stationarity should be checked at alpha = 0.05.

To check the stationarity of the data, first we need to perform Dickey Fuller test

Dickey Fuller Test:

For this test, we need to make null and alternate hypothesis:

We will take the Null Hypothesis H_0 as - Series is not Stationary.

and Alternative Hypothesis H_1 as - Series is Stationary.

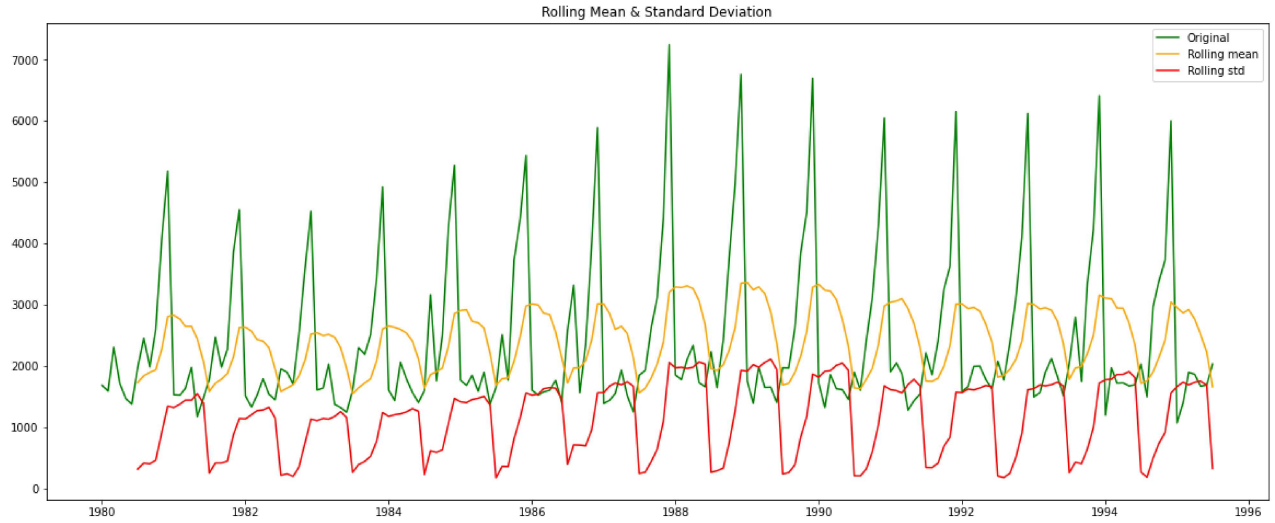
Since We have to proof that data is stationary and also we will take steps to make the data stationary.

In [75]:
```
#importing adfuller

from statsmodels.tsa.stattools import adfuller
```

In [76]:
```
#creating a function to test the stationarity of the time series


def Stationarity_test(timeseries):

    #rolling mean
    rolmean = timeseries.rolling(window=7).mean()
    #rolling standard deviation
    rolstd = timeseries.rolling(window=7).std()

    #Plotting

    original = plt.plot(timeseries, color='green',label='Original')
    mean = plt.plot(rolmean, color='orange', label='Rolling mean')
    std = plt.plot(rolstd, color='red', label = 'Rolling std')
    plt.legend(loc='best')
    plt.title('Rolling Mean & Standard Deviation')
    plt.show(block=False)
```

In [77]:
```
#calling the function
```

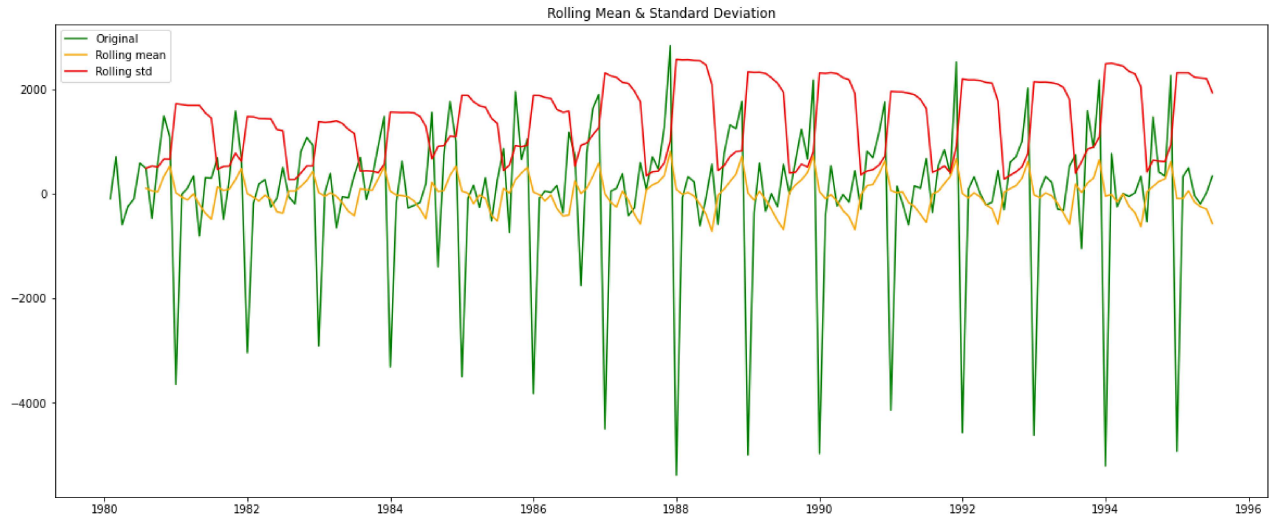
```
Stationarity_test(sparkling_data['Sparkling'])
```



Time series does not seems to be stationary at 5% significant level.

We have to check again with the difference of order 1

In [78]:
```
Stationarity_test(sparkling_data['Sparkling'].diff().dropna())
```



Now the series is stationary.

6. Build an automated version of the ARIMA/SARIMA model in which the parameters are selected using the lowest Akaike Information Criteria (AIC) on the training data and evaluate this model on the test data using RMSE.

In [79]:
```
#we will take the combination of different parameters of p and q in the range of 0 and

#we have kept the value of d as 1 as we need to take a difference of the series to make

#importing itertools

import itertools
p = q = range(0, 3)
```

```
d= range(1,2)
pdq = list(itertools.product(p, d, q))
print('Combination of parameters for the model:')
for i in range(1,len(pdq)):
    print('Model: {}'.format(pdq[i]))
```

```
Combination of parameters for the model:
Model: (0, 1, 1)
Model: (0, 1, 2)
Model: (1, 1, 0)
Model: (1, 1, 1)
Model: (1, 1, 2)
Model: (2, 1, 0)
Model: (2, 1, 1)
Model: (2, 1, 2)
```

In [80]:

```
# Creating an empty Dataframe with column names only
ARIMA_AIC = pd.DataFrame(columns=['param', 'AIC'])
ARIMA_AIC
```

Out[80]:

	param	AIC

In [81]:

```
#importing ARIMA

from statsmodels.tsa.arima_model import ARIMA
```

In [82]:

```
for param in pdq:
    ARIMA_model = ARIMA(train['Sparkling'].values,order=param).fit()
    print('ARIMA{} - AIC:{}'.format(param,ARIMA_model.aic))
    ARIMA_AIC = ARIMA_AIC.append({'param':param, 'AIC': ARIMA_model.aic}, ignore_index=
```

```
ARIMA(0, 1, 0) - AIC:2269.582796371201
ARIMA(0, 1, 1) - AIC:2264.906438611577
ARIMA(0, 1, 2) - AIC:2232.78309768405
ARIMA(1, 1, 0) - AIC:2268.5280607650034
ARIMA(1, 1, 1) - AIC:2235.0139453493175
ARIMA(1, 1, 2) - AIC:2233.597647121205
ARIMA(2, 1, 0) - AIC:2262.035600155025
ARIMA(2, 1, 1) - AIC:2232.3604899121387
ARIMA(2, 1, 2) - AIC:2210.6260491378257
```

The lowest AIC value from the above evaluated parameters is (2,1,2) that is 2210.62

In [83]:

```
#Now, let's create auto ARIMA by using these parameters

auto_ARIMA = ARIMA(train['Sparkling'], order=(2,1,2))
results_auto_ARIMA = auto_ARIMA.fit()
print(results_auto_ARIMA.summary())
```

```
                             ARIMA Model Results
==============================================================================
Dep. Variable:            D.Sparkling   No. Observations:                  131
Model:                 ARIMA(2, 1, 2)   Log Likelihood               -1099.313
Method:                       css-mle   S.D. of innovations            1013.755
Date:                Fri, 08 Oct 2021   AIC                            2210.626
Time:                        18:54:49   BIC                            2227.877
```

```
Sample:                  02-01-1980   HQIC                          2217.636
                       - 12-01-1990
==============================================================================
                     coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const                5.5845      0.519     10.753      0.000       4.567       6.602
ar.L1.D.Sparkling    1.2698      0.075     17.040      0.000       1.124       1.416
ar.L2.D.Sparkling   -0.5601      0.074     -7.617      0.000      -0.704      -0.416
ma.L1.D.Sparkling   -1.9957      0.043    -46.821      0.000      -2.079      -1.912
ma.L2.D.Sparkling    0.9957      0.043     23.291      0.000       0.912       1.079
                                    Roots
=============================================================================
                  Real          Imaginary           Modulus         Frequency
-----------------------------------------------------------------------------
AR.1            1.1335           -0.7074j            1.3361           -0.0888
AR.2            1.1335           +0.7074j            1.3361            0.0888
MA.1            1.0000           +0.0000j            1.0000            0.0000
MA.2            1.0043           +0.0000j            1.0043            0.0000
-----------------------------------------------------------------------------
```

In [84]:
```
#prediction on test set

predicted_auto_ARIMA = results_auto_ARIMA.forecast(steps=len(test))
```

In [85]:
```
#importing mean_squared_error

from sklearn.metrics import  mean_squared_error
```

In [86]:
```
#Calculating RMSE on test data

RMSE = mean_squared_error(test['Sparkling'],predicted_auto_ARIMA[0],squared=False)
```

In [87]:
```
print(RMSE)
```

```
1374.0370092580251
```

In [88]:
```
#Comparing and making dataframe

resultsDf_8 = pd.DataFrame({'Test RMSE': [RMSE]}
                           ,index=['ARIMA (2,1,2)'])


resultsDf = pd.concat([resultsDf, resultsDf_8])
```

In [89]:
```
resultsDf
```

Out[89]:

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804

	Test RMSE
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
ARIMA (2,1,2)	1374.037009

Automated SARIMA

In [90]:
```
import itertools
p = q = range(0, 3)
d= range(1,2)
D = range(0,1)
pdq = list(itertools.product(p, d, q))
model_pdq = [(x[0], x[1], x[2], 12) for x in list(itertools.product(p, D, q))]
print('Combinations of parameter for the SARIMA model:')
for i in range(1,len(pdq)):
    print('Model: {}{}'.format(pdq[i], model_pdq[i]))
```

```
Combinations of parameter for the SARIMA model:
Model: (0, 1, 1)(0, 0, 1, 12)
Model: (0, 1, 2)(0, 0, 2, 12)
Model: (1, 1, 0)(1, 0, 0, 12)
Model: (1, 1, 1)(1, 0, 1, 12)
Model: (1, 1, 2)(1, 0, 2, 12)
Model: (2, 1, 0)(2, 0, 0, 12)
Model: (2, 1, 1)(2, 0, 1, 12)
Model: (2, 1, 2)(2, 0, 2, 12)
```

In [91]:
```
SARIMA_AIC = pd.DataFrame(columns=['param','seasonal', 'AIC'])
SARIMA_AIC
```

Out[91]:

	param	seasonal	AIC

In [92]:
```
import statsmodels.api as sm

for param in pdq:
    for param_seasonal in model_pdq:
        SARIMA_model = sm.tsa.statespace.SARIMAX(train['Sparkling'].values,
                                                 order=param,
                                                 seasonal_order=param_seasonal,
                                                 enforce_stationarity=False,
                                                 enforce_invertibility=False)

        results_SARIMA = SARIMA_model.fit(maxiter=1000)
        print('SARIMA{}x{} - AIC:{}'.format(param, param_seasonal, results_SARIMA.aic))
        SARIMA_AIC = SARIMA_AIC.append({'param':param,'seasonal':param_seasonal ,'AIC':
```

```
SARIMA(0, 1, 0)x(0, 0, 0, 12) - AIC:2251.3597196862966
SARIMA(0, 1, 0)x(0, 0, 1, 12) - AIC:1956.2614616842675
SARIMA(0, 1, 0)x(0, 0, 2, 12) - AIC:1723.1533640235496
SARIMA(0, 1, 0)x(1, 0, 0, 12) - AIC:1837.4366022456675
SARIMA(0, 1, 0)x(1, 0, 1, 12) - AIC:1806.9905301389083
SARIMA(0, 1, 0)x(1, 0, 2, 12) - AIC:1633.2108735791944
SARIMA(0, 1, 0)x(2, 0, 0, 12) - AIC:1648.3776153470856
SARIMA(0, 1, 0)x(2, 0, 1, 12) - AIC:1647.2054158611443
SARIMA(0, 1, 0)x(2, 0, 2, 12) - AIC:1630.9898053920774
SARIMA(0, 1, 1)x(0, 0, 0, 12) - AIC:2230.1629078505853
SARIMA(0, 1, 1)x(0, 0, 1, 12) - AIC:1923.768864956687
SARIMA(0, 1, 1)x(0, 0, 2, 12) - AIC:1692.708957278316
SARIMA(0, 1, 1)x(1, 0, 0, 12) - AIC:1797.1795881838032
SARIMA(0, 1, 1)x(1, 0, 1, 12) - AIC:1738.090319373017
SARIMA(0, 1, 1)x(1, 0, 2, 12) - AIC:1570.13196585489
SARIMA(0, 1, 1)x(2, 0, 0, 12) - AIC:1605.6751954176586
SARIMA(0, 1, 1)x(2, 0, 1, 12) - AIC:1599.2245088165164
SARIMA(0, 1, 1)x(2, 0, 2, 12) - AIC:1570.4018833130494
SARIMA(0, 1, 2)x(0, 0, 0, 12) - AIC:2187.4410101687017
SARIMA(0, 1, 2)x(0, 0, 1, 12) - AIC:1887.9128007202285
SARIMA(0, 1, 2)x(0, 0, 2, 12) - AIC:1659.8789890444405
SARIMA(0, 1, 2)x(1, 0, 0, 12) - AIC:1790.0326332162715
SARIMA(0, 1, 2)x(1, 0, 1, 12) - AIC:1724.1675077971956
SARIMA(0, 1, 2)x(1, 0, 2, 12) - AIC:1557.1605065895915
SARIMA(0, 1, 2)x(2, 0, 0, 12) - AIC:1603.9654774499593
SARIMA(0, 1, 2)x(2, 0, 1, 12) - AIC:1600.5438800921174
SARIMA(0, 1, 2)x(2, 0, 2, 12) - AIC:1557.1215631487498
SARIMA(1, 1, 0)x(0, 0, 0, 12) - AIC:2250.3181267386713
SARIMA(1, 1, 0)x(0, 0, 1, 12) - AIC:1954.3938339903718
SARIMA(1, 1, 0)x(0, 0, 2, 12) - AIC:1721.26884763557
SARIMA(1, 1, 0)x(1, 0, 0, 12) - AIC:1811.2440279330933
SARIMA(1, 1, 0)x(1, 0, 1, 12) - AIC:1788.5343592679392
SARIMA(1, 1, 0)x(1, 0, 2, 12) - AIC:1616.4894402560076
SARIMA(1, 1, 0)x(2, 0, 0, 12) - AIC:1621.6355080129133
SARIMA(1, 1, 0)x(2, 0, 1, 12) - AIC:1617.135613205786
SARIMA(1, 1, 0)x(2, 0, 2, 12) - AIC:1616.5412067647578
SARIMA(1, 1, 1)x(0, 0, 0, 12) - AIC:2204.934049154559
SARIMA(1, 1, 1)x(0, 0, 1, 12) - AIC:1907.35589741287
SARIMA(1, 1, 1)x(0, 0, 2, 12) - AIC:1678.0981352624256
SARIMA(1, 1, 1)x(1, 0, 0, 12) - AIC:1775.1424466615247
SARIMA(1, 1, 1)x(1, 0, 1, 12) - AIC:1739.7167467829483
SARIMA(1, 1, 1)x(1, 0, 2, 12) - AIC:1571.3248864017028
SARIMA(1, 1, 1)x(2, 0, 0, 12) - AIC:1590.6161606872656
SARIMA(1, 1, 1)x(2, 0, 1, 12) - AIC:1586.3140614367558
SARIMA(1, 1, 1)x(2, 0, 2, 12) - AIC:1571.8069968641123
SARIMA(1, 1, 2)x(0, 0, 0, 12) - AIC:2188.4633450504807
SARIMA(1, 1, 2)x(0, 0, 1, 12) - AIC:1889.7708307504445
SARIMA(1, 1, 2)x(0, 0, 2, 12) - AIC:1659.6291421495487
SARIMA(1, 1, 2)x(1, 0, 0, 12) - AIC:1771.8259799530147
SARIMA(1, 1, 2)x(1, 0, 1, 12) - AIC:1723.9871793212103
SARIMA(1, 1, 2)x(1, 0, 2, 12) - AIC:1555.5842468448625
SARIMA(1, 1, 2)x(2, 0, 0, 12) - AIC:1588.4216931923386
SARIMA(1, 1, 2)x(2, 0, 1, 12) - AIC:1585.5152937655243
SARIMA(1, 1, 2)x(2, 0, 2, 12) - AIC:1555.934562653481
SARIMA(2, 1, 0)x(0, 0, 0, 12) - AIC:2227.302761872421
SARIMA(2, 1, 0)x(0, 0, 1, 12) - AIC:1946.4383435408604
SARIMA(2, 1, 0)x(0, 0, 2, 12) - AIC:1711.4123039822166
SARIMA(2, 1, 0)x(1, 0, 0, 12) - AIC:1780.7646066059465
SARIMA(2, 1, 0)x(1, 0, 1, 12) - AIC:1756.935734715416
SARIMA(2, 1, 0)x(1, 0, 2, 12) - AIC:1600.9702203985114
SARIMA(2, 1, 0)x(2, 0, 0, 12) - AIC:1592.2403464860076
```

```
SARIMA(2, 1, 0)x(2, 0, 1, 12) - AIC:1587.6344991504284
SARIMA(2, 1, 0)x(2, 0, 2, 12) - AIC:1585.919173227418
SARIMA(2, 1, 1)x(0, 0, 0, 12) - AIC:2199.858613145789
SARIMA(2, 1, 1)x(0, 0, 1, 12) - AIC:1905.0209495219829
SARIMA(2, 1, 1)x(0, 0, 2, 12) - AIC:1675.4234080320236
SARIMA(2, 1, 1)x(1, 0, 0, 12) - AIC:1792.8234291874323
SARIMA(2, 1, 1)x(1, 0, 1, 12) - AIC:1740.0911276667364
SARIMA(2, 1, 1)x(1, 0, 2, 12) - AIC:1571.9888282546003
SARIMA(2, 1, 1)x(2, 0, 0, 12) - AIC:1577.123506089688
SARIMA(2, 1, 1)x(2, 0, 1, 12) - AIC:1573.1595857703485
SARIMA(2, 1, 1)x(2, 0, 2, 12) - AIC:1572.3428643854995
SARIMA(2, 1, 2)x(0, 0, 0, 12) - AIC:2176.8681149359595
SARIMA(2, 1, 2)x(0, 0, 1, 12) - AIC:1892.2372612883312
SARIMA(2, 1, 2)x(0, 0, 2, 12) - AIC:1661.5523433339908
SARIMA(2, 1, 2)x(1, 0, 0, 12) - AIC:1757.2140931211306
SARIMA(2, 1, 2)x(1, 0, 1, 12) - AIC:1725.6005734246855
SARIMA(2, 1, 2)x(1, 0, 2, 12) - AIC:1557.3404035073365
SARIMA(2, 1, 2)x(2, 0, 0, 12) - AIC:1625.2273641149545
SARIMA(2, 1, 2)x(2, 0, 1, 12) - AIC:1573.5476069401338
SARIMA(2, 1, 2)x(2, 0, 2, 12) - AIC:1557.689323020058
```

In [93]:

```
#Let's check top 5 lowest AIC values

SARIMA_AIC.sort_values(by=['AIC']).head()
```

Out[93]:

	param	seasonal	AIC
50	(1, 1, 2)	(1, 0, 2, 12)	1555.584247
53	(1, 1, 2)	(2, 0, 2, 12)	1555.934563
26	(0, 1, 2)	(2, 0, 2, 12)	1557.121563
23	(0, 1, 2)	(1, 0, 2, 12)	1557.160507
77	(2, 1, 2)	(1, 0, 2, 12)	1557.340404

In [94]:

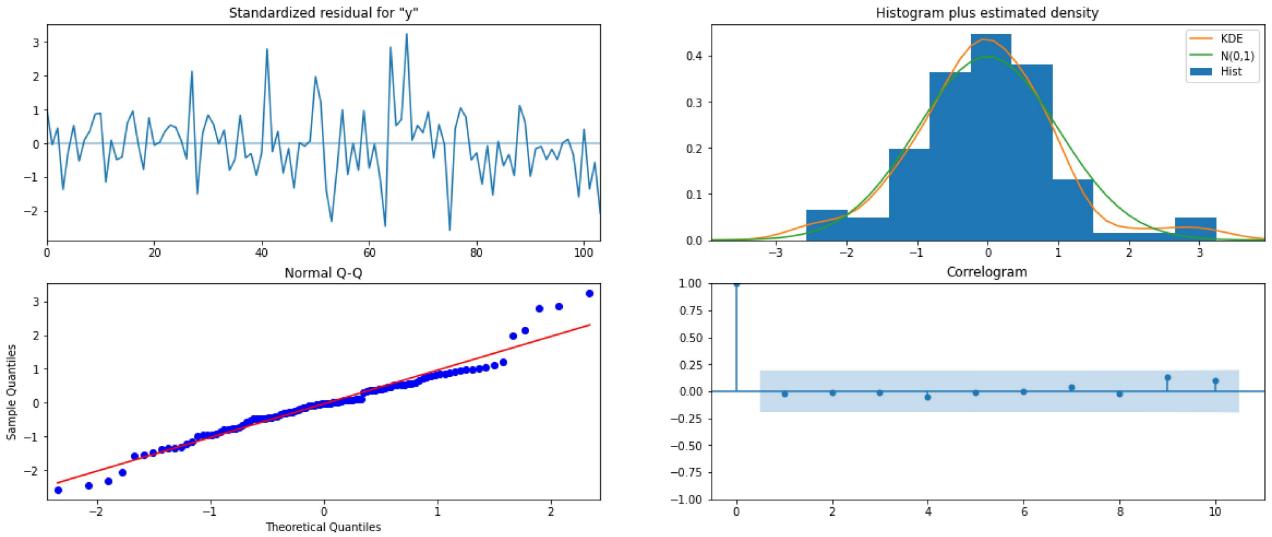
```
import statsmodels.api as sm

auto_SARIMA_12 = sm.tsa.statespace.SARIMAX(train['Sparkling'].values,
                                order=(1, 1, 2),
                                seasonal_order=(1, 0, 2, 12),
                                enforce_stationarity=False,
                                enforce_invertibility=False)
results_auto_SARIMA_12 = auto_SARIMA_12.fit(maxiter=1000)
print(results_auto_SARIMA_12.summary())
```

```
                                     SARIMAX Results
==========================================================================================
==
Dep. Variable:                                   y   No. Observations:                1
32
Model:             SARIMAX(1, 1, 2)x(1, 0, 2, 12)   Log Likelihood                -770.7
92
Date:                            Fri, 08 Oct 2021   AIC                            1555.5
84
Time:                                    18:56:38   BIC                            1574.0
95
Sample:                                         0   HQIC                           1563.0
```

```
83
                                                - 132
Covariance Type:                                opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.6282      0.255     -2.463      0.014      -1.128      -0.128
ma.L1         -0.1041      0.225     -0.463      0.643      -0.545       0.337
ma.L2         -0.7276      0.154     -4.735      0.000      -1.029      -0.426
ar.S.L12       1.0439      0.014     72.840      0.000       1.016       1.072
ma.S.L12      -0.5550      0.098     -5.663      0.000      -0.747      -0.363
ma.S.L24      -0.1354      0.120     -1.133      0.257      -0.370       0.099
sigma2     1.506e+05   2.03e+04      7.401      0.000    1.11e+05     1.9e+05
===================================================================================
Ljung-Box (L1) (Q):                   0.04   Jarque-Bera (JB):                11.72
Prob(Q):                              0.84   Prob(JB):                         0.00
Heteroskedasticity (H):               1.47   Skew:                             0.36
Prob(H) (two-sided):                  0.26   Kurtosis:                         4.48
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

In [95]:

```
#Plotting

results_auto_SARIMA_12.plot_diagnostics()
plt.show()
```



In [96]:

```
#prediction on test set

predicted_auto_SARIMA_12 = results_auto_SARIMA_12.get_forecast(steps=len(test))
predicted_auto_SARIMA_12.summary_frame(alpha=0.05).head() #for alpha = 0.05
```

Out[96]:

	y	mean	mean_se	mean_ci_lower	mean_ci_upper
0		1327.391808	388.342295	566.254897	2088.528719
1		1315.124183	402.004779	527.209295	2103.039070
2		1621.595424	401.998386	833.693067	2409.497782

y	mean	mean_se	mean_ci_lower	mean_ci_upper
3	1598.879289	407.235895	800.711601	2397.046976
4	1392.696749	407.965802	593.098470	2192.295028

In [97]:
```
#Calculating RMSE on test data

rmse = mean_squared_error(test['Sparkling'],predicted_auto_SARIMA_12.predicted_mean,squ

print(rmse)
```

```
528.6126533320019
```

In [98]:
```
#comparing and displaying all the RMSE values

resultsDf_9 = pd.DataFrame({'Test RMSE': [rmse]}
                           ,index=['SARIMA (1,1,2)(1,0,2,12)'])
resultsDf = pd.concat([resultsDf, resultsDf_9])
resultsDf
```

Out[98]:

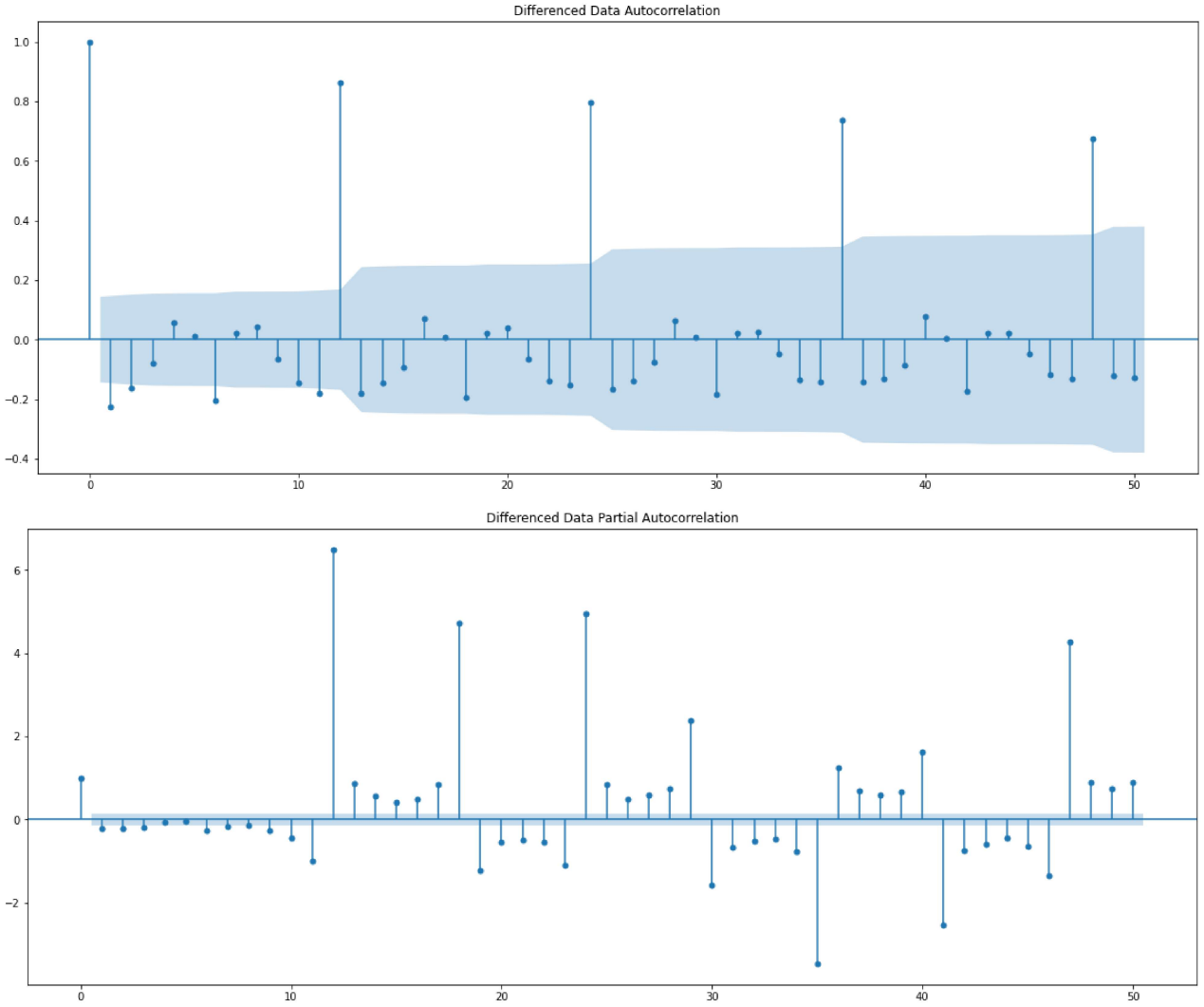
	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
ARIMA (2,1,2)	1374.037009
SARIMA (1,1,2)(1,0,2,12)	528.612653

Till now the TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395) has the best RMSE value that is 469.767970

7. Build ARIMA/SARIMA models based on the cut-off points of ACF and PACF on the training data and evaluate this model on the test data using RMSE.

In [99]:
```
#Let's plot the ARIMA/SARIMA models with cut off points of ACF and PACF on training dat

#import plot_acf and plot_pacf

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
plot_acf(sparkling_data['Sparkling'].diff().dropna(),lags=50,title='Differenced Data Au
plot_pacf(sparkling_data['Sparkling'].diff().dropna(),lags=50,title='Differenced Data P
plt.show()
```



PACF cut offs point at lag 3 and ACF cut offs point at lag 2

In [100…

```
manual_ARIMA = ARIMA(train['Sparkling'].astype('float64'), order=(3,1,2))
results_manual_ARIMA = manual_ARIMA.fit()
print(results_manual_ARIMA.summary())
```

```
                             ARIMA Model Results
==============================================================================
Dep. Variable:            D.Sparkling   No. Observations:                  131
Model:                 ARIMA(3, 1, 2)   Log Likelihood               -1107.464
Method:                       css-mle   S.D. of innovations           1106.238
Date:                Fri, 08 Oct 2021   AIC                           2228.928
Time:                        18:56:42   BIC                           2249.054
Sample:                    02-01-1980   HQIC                          2237.106
                         - 12-01-1990
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          5.9844      3.643      1.643      0.100      -1.156      13.125
```

```
ar.L1.D.Sparkling    -0.4420   1.28e-05  -3.46e+04      0.000      -0.442      -0.442
ar.L2.D.Sparkling     0.3079   4.63e-05   6645.786      0.000       0.308       0.308
ar.L3.D.Sparkling    -0.2501   3.81e-05  -6560.861      0.000      -0.250      -0.250
ma.L1.D.Sparkling    -0.0008      0.020     -0.040      0.968      -0.039       0.037
ma.L2.D.Sparkling    -0.9992      0.020    -51.220      0.000      -1.037      -0.961
                                    Roots
=============================================================================
                  Real          Imaginary           Modulus         Frequency
-----------------------------------------------------------------------------
AR.1           -1.0000           -0.0000j            1.0000           -0.5000
AR.2            1.1156           -1.6594j            1.9996           -0.1558
AR.3            1.1156           +1.6594j            1.9996            0.1558
MA.1            1.0000           +0.0000j            1.0000            0.0000
MA.2           -1.0008           +0.0000j            1.0008            0.5000
-----------------------------------------------------------------------------
```

In [101…

```
predicted_manual_ARIMA = results_manual_ARIMA.forecast(steps=len(test))
```

In [102…

```
#RMSE

rmse = mean_squared_error(test['Sparkling'],predicted_manual_ARIMA[0],squared=False)
print(rmse)
```

```
1379.0491231340152
```

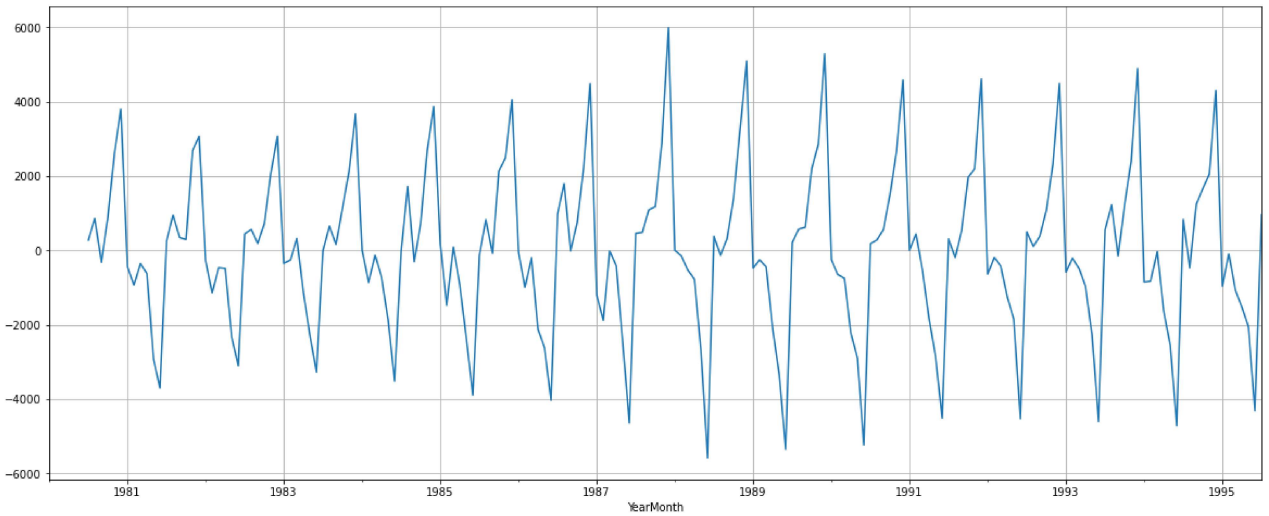
In [103…

```
#comparing all rmse

resultsDf_10 = pd.DataFrame({'Test RMSE': [rmse]}
                           ,index=['ARIMA (3,1,2) based on ACF & PACF'])

resultsDf = pd.concat([resultsDf, resultsDf_10])

resultsDf
```

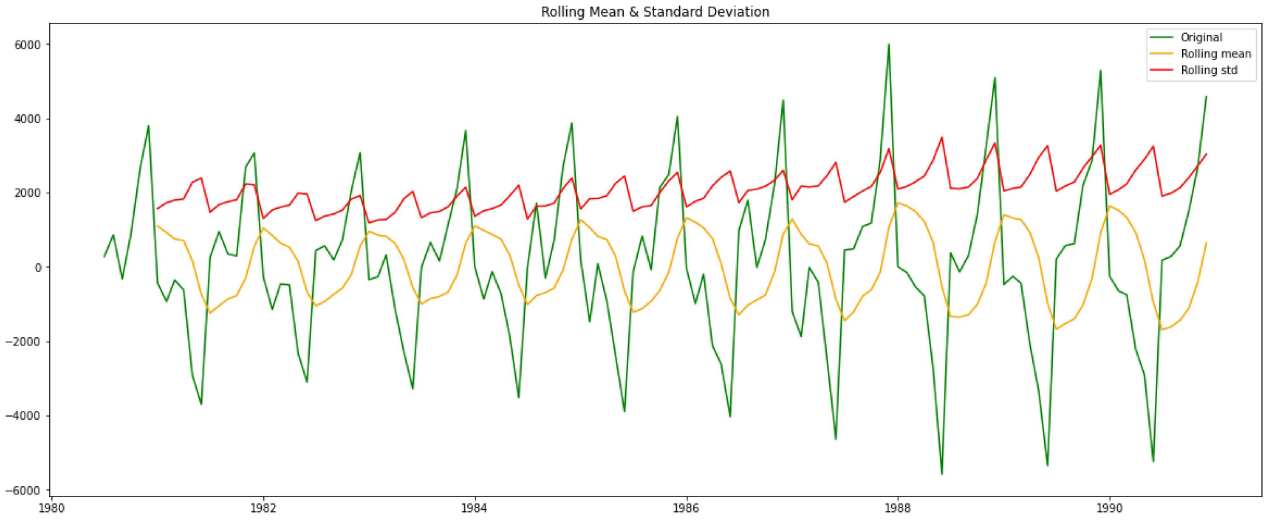
Out[103…

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
ARIMA (2,1,2)	1374.037009
SARIMA (1,1,2)(1,0,2,12)	528.612653
ARIMA (3,1,2) based on ACF & PACF	1379.049123

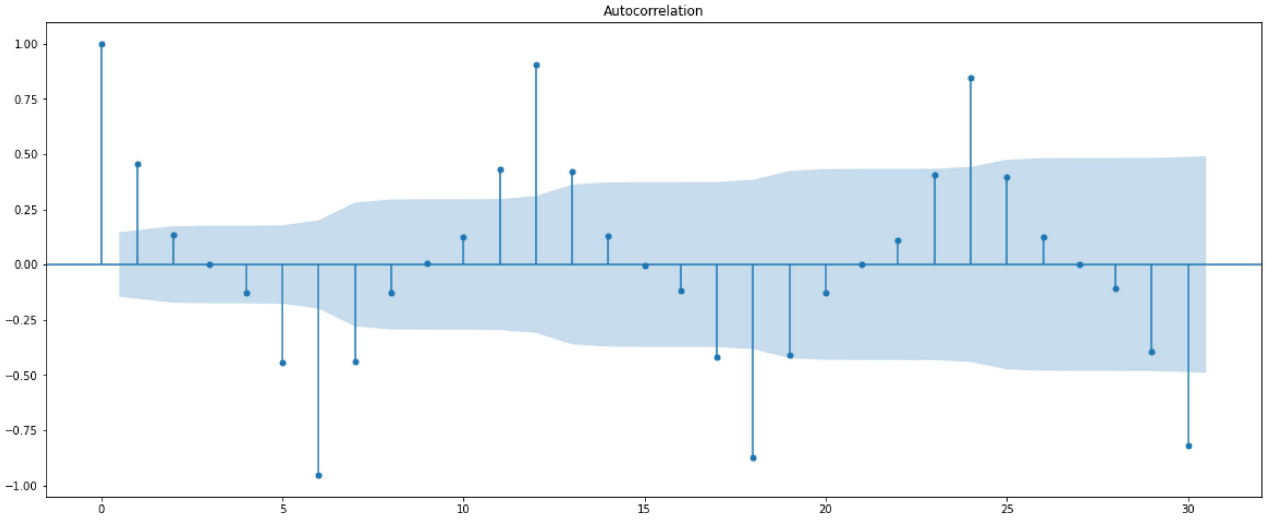
In [104…
```
#at differnecing of 6, let's make a plot

(sparkling_data['Sparkling'].diff(6)).plot()
plt.grid();
```

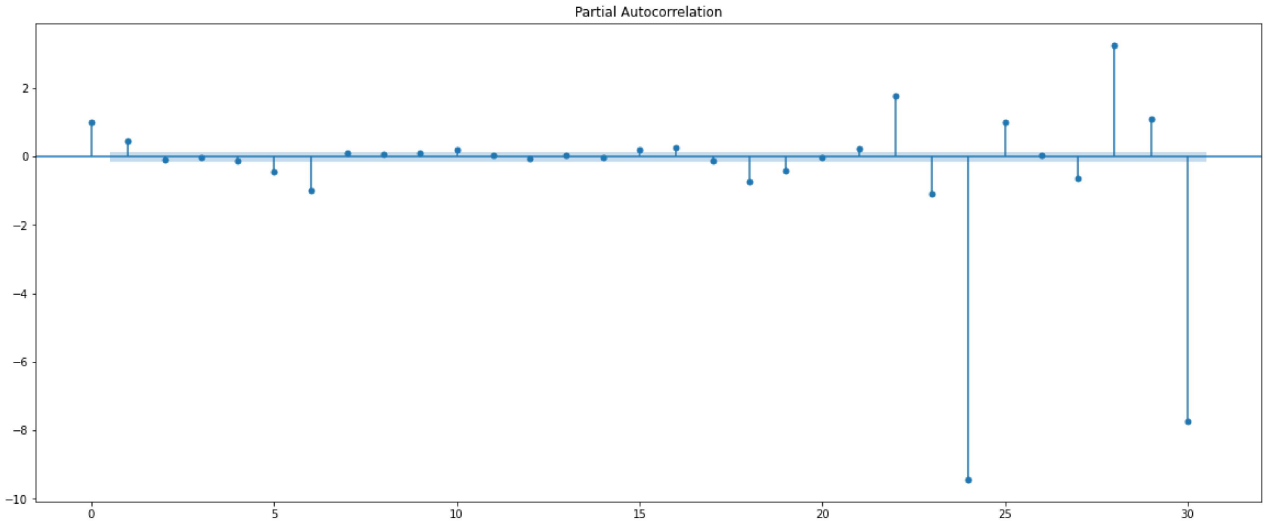


In [105…
```
#Again checking stationary of the series using our function Stationary_test

Stationarity_test(train['Sparkling'].diff(6).dropna())
```



In [106…
```
#again plotting with taking the seasonal period as 6

plot_acf((sparkling_data['Sparkling'].diff(6).dropna()),lags=30)
plot_pacf((sparkling_data['Sparkling'].diff(6).dropna()),lags=30);
```





Now, both the ACF and PACF plots have the cut off at 1.

Alpha value at 0.05.

we will check the ACF and PACF plots at a multiple of 6, because seasonal period is 6

In [107…

```
manual_SARIMA_model = sm.tsa.statespace.SARIMAX(train['Sparkling'].values,
                                order=(3, 1, 2),
                                seasonal_order=(1, 1, 1, 6),
                                enforce_stationarity=False,
                                enforce_invertibility=False)
results_manual_SARIMA_model = manual_SARIMA_model.fit(maxiter=1000)
```

In [108…

```
print(results_manual_SARIMA_model.summary())
```

```
                                     SARIMAX Results
==========================================================================================
===
Dep. Variable:                                     y   No. Observations:
132
Model:             SARIMAX(3, 1, 2)x(1, 1, [1], 6)   Log Likelihood                -866.
759
Date:                               Fri, 08 Oct 2021   AIC                           1749.
```

```
519
Time:                                  18:56:47   BIC                            1771.
547
Sample:                                    0      HQIC                           1758.
461
                                         - 132
Covariance Type:                           opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.0416      0.896     -0.046      0.963      -1.798       1.715
ar.L2         -0.0162      0.272     -0.060      0.953      -0.548       0.516
ar.L3          0.0884      0.140      0.630      0.529      -0.187       0.363
ma.L1         -0.6756      0.956     -0.706      0.480      -2.550       1.199
ma.L2         -0.3244      0.884     -0.367      0.714      -2.057       1.408
ar.S.L6       -0.9784      0.025    -39.614      0.000      -1.027      -0.930
ma.S.L6       -0.1195      0.132     -0.907      0.365      -0.378       0.139
sigma2      1.746e+05   3.86e-06   4.53e+10      0.000    1.75e+05    1.75e+05
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                10.80
Prob(Q):                              0.93   Prob(JB):                         0.00
Heteroskedasticity (H):               1.72   Skew:                             0.23
Prob(H) (two-sided):                  0.09   Kurtosis:                         4.42
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 7.73e+26. Stan
dard errors may be unstable.
```

Predict on the Test Set using this model and evaluate the model.

In [109…
```
predicted_manual_SARIMA_model = results_manual_SARIMA_model.get_forecast(steps=len(test
```

In [110…
```
#at aplha 0.05

predicted_manual_SARIMA_model.summary_frame(alpha=0.05).head()
```

Out[110…

y	mean	mean_se	mean_ci_lower	mean_ci_upper
0	1529.776845	419.586764	707.401900	2352.151791
1	1386.732195	436.989129	530.249240	2243.215150
2	1863.791987	437.026609	1007.235573	2720.348400
3	1741.536788	438.833155	881.439608	2601.633968
4	1712.828898	439.027078	852.351637	2573.306158

In [111…
```
#RMSE on test data

rmse = mean_squared_error(test['Sparkling'],predicted_manual_SARIMA_model.predicted_mea
```

In [112…
```
print(rmse)
```

```
329.88649987548496
```

In [113…

```
#Comparing all rmse

resultsDf_11 = pd.DataFrame({'Test RMSE': [rmse]}
                            ,index=['SARIMA(3,1,2)(1,1,1,6) based on ACF & PACF'])
resultsDf = pd.concat([resultsDf, resultsDf_11])
resultsDf
```

Out[113…

	Test RMSE
Regression_on_time	1389.135175
NaiveModel	3864.279352
Simple_Average_Model	1275.081804
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
6pointTrailingMovingAverage	1283.927428
9pointTrailingMovingAverage	1346.278315
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
ARIMA (2,1,2)	1374.037009
SARIMA (1,1,2)(1,0,2,12)	528.612653
ARIMA (3,1,2) based on ACF & PACF	1379.049123
SARIMA(3,1,2)(1,1,1,6) based on ACF & PACF	329.886500

8. Build a table with all the models built along with their corresponding parameters and the respective RMSE values on the test data.

In [114…

```
#A table with all the RMSE values along with all the corresponding parameters :on test

#values sorted in ascending order

resultsDf.sort_values(by=['Test RMSE'])
```

Out[114…

	Test RMSE
SARIMA(3,1,2)(1,1,1,6) based on ACF & PACF	329.886500
TripleExponentialSmoothing (for Alpha=0.111,Beta=0.061,Gamma=0.395)	469.767970
SARIMA (1,1,2)(1,0,2,12)	528.612653
2pointTrailingMovingAverage	813.400684
4pointTrailingMovingAverage	1156.589694
Simple_Average_Model	1275.081804

	Test RMSE
6pointTrailingMovingAverage	1283.927428
SimpleExponentialSmoothing (for alpha 0.070)	1338.008384
9pointTrailingMovingAverage	1346.278315
ARIMA (2,1,2)	1374.037009
ARIMA (3,1,2) based on ACF & PACF	1379.049123
Regression_on_time	1389.135175
NaiveModel	3864.279352

We can see that SARIMA model is best model with the lowest RMSE value of 329.88

9. Based on the model-building exercise, build the most optimum model(s) on the complete data and predict 12 months into the future with appropriate confidence intervals/bands.

Now we have to build the most optimum model on full dataset

In [115…

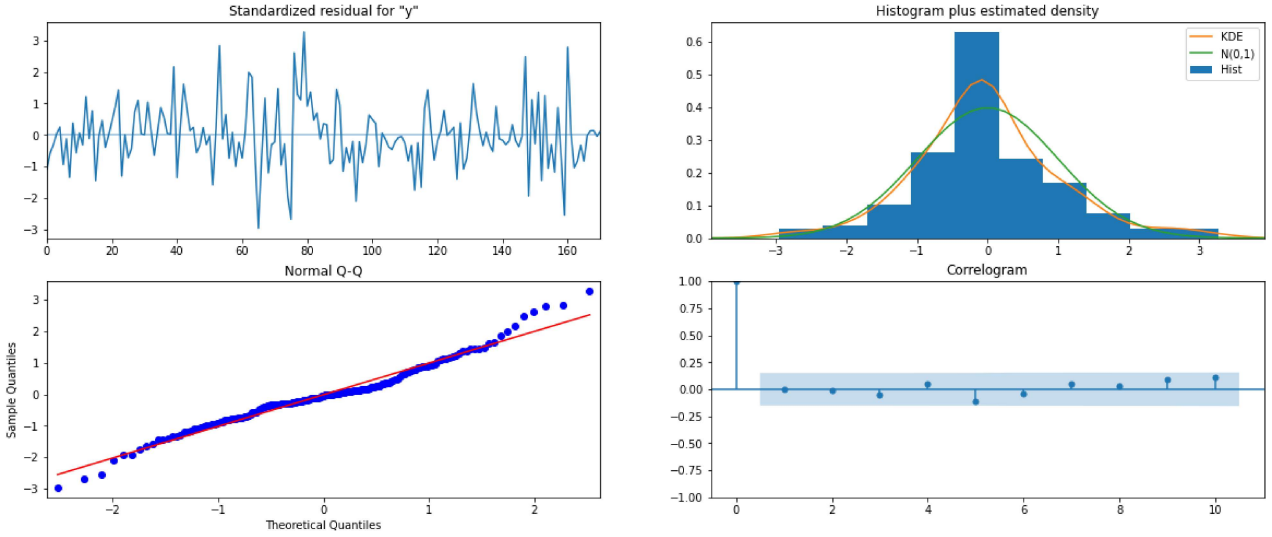
```
optimum_model = sm.tsa.statespace.SARIMAX(sparkling_data['Sparkling'].values,
                                order=(3,1,2),
                                seasonal_order=(1,1,1,6),
                                enforce_stationarity=False,
                                enforce_invertibility=False)
results_optimum_model = optimum_model.fit(maxiter=1000)
print(results_optimum_model.summary())
```

```
                                     SARIMAX Results
==========================================================================================
===
Dep. Variable:                                     y   No. Observations:
187
Model:             SARIMAX(3, 1, 2)x(1, 1, [1], 6)   Log Likelihood               -1274.
645
Date:                             Fri, 08 Oct 2021   AIC                           2565.
290
Time:                                     18:56:51   BIC                           2590.
424
Sample:                                          0   HQIC                          2575.
488
                                               - 187
Covariance Type:                                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.8299      0.066    -12.656      0.000      -0.958      -0.701
ar.L2          0.0224      0.077      0.290      0.772      -0.129       0.174
ar.L3         -0.0101      0.070     -0.145      0.884      -0.147       0.126
ma.L1      -2.344e-05     18.739  -1.25e-06      1.000     -36.728      36.728
ma.L2         -1.0000      0.095    -10.550      0.000      -1.186      -0.814
ar.S.L6       -0.9906      0.014    -68.653      0.000      -1.019      -0.962
ma.S.L6        0.1700      0.094      1.800      0.072      -0.015       0.355
sigma2      1.667e+05   9.77e-05   1.71e+09      0.000    1.67e+05    1.67e+05
```

```
================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                12.01
Prob(Q):                              0.96   Prob(JB):                         0.00
Heteroskedasticity (H):               1.28   Skew:                             0.31
Prob(H) (two-sided):                  0.35   Kurtosis:                         4.14
================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.96e+26. Stan
dard errors may be unstable.
```

In [116…

```
#Diagnostics

results_optimum_model.plot_diagnostics();
```



In [117…

```
#evaluation of the model on full dataset

#as given in the question, we have to predict 12 months in the future

predicted_manual_SARIMA_model_optimum = results_optimum_model.get_forecast(steps=12)
```

In [118…

```
predicted_manual_SARIMA_model_optimum.summary_frame(alpha=0.05).head()
```

Out[118…

y	mean	mean_se	mean_ci_lower	mean_ci_upper
0	1474.396779	410.603867	669.627988	2279.165570
1	2938.141684	416.494564	2121.827338	3754.456030
2	3416.258831	418.823951	2595.378972	4237.138691
3	3711.960077	420.594649	2887.609714	4536.310441
4	5986.150249	421.599368	5159.830671	6812.469827

In [119…

```
#RMSE
```

```
rmse = mean_squared_error(sparkling_data['Sparkling'],results_optimum_model.fittedvalue
print('RMSE of the best optimum model on whole dataset is:',rmse)
```

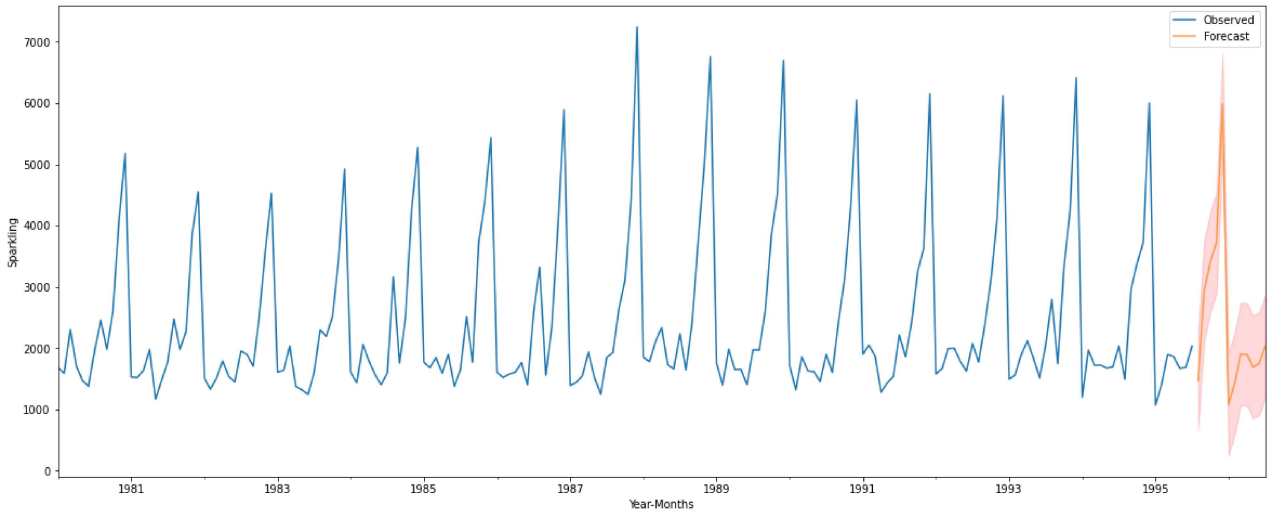
```
RMSE of the best optimum model on whole dataset is: 571.8691917375559
```

In [120…

```
pred_full_manual_SARIMA_date = predicted_manual_SARIMA_model_optimum.summary_frame(alph
```

In [121…

```
#plotting

axis = sparkling_data['Sparkling'].plot(label='Observed')
pred_full_manual_SARIMA_date['mean'].plot(ax=axis, label='Forecast', alpha=0.7)
axis.fill_between(pred_full_manual_SARIMA_date.index, pred_full_manual_SARIMA_date['mea
                  pred_full_manual_SARIMA_date['mean_ci_upper'], color='r', alpha=.15)
axis.set_xlabel('Year-Months')
axis.set_ylabel('Sparkling')
plt.legend(loc='best')
plt.show()
```



10.Comment on the model thus built and report your findings and suggest the measures that the company should be taking for future sales.

Some Measures that company need to take care of in future:

We have observed the highest Sales happened in the year 1987, and the lowest sales observed in the year 1995. From the decomposition plot, we saw that the trend and seasonality are present in the dataset. We have analyzed from boxplot (monthly plot) that sales has been increased from month of September to month of December. We have built various models and tested on test data and also calculated RMSE values on test data to predict the forecast for next 12 months or more. We have also observed that the original dataset has some seasonality pattern. Found out that SARIMA has the lowest RMSE value. So the same model later was applied on full data. On the basis of final and most optimum model, the sales for next 12 month is predicted with confidence interval. Company should keep its focus on reserved stock during the peak time, and in January, the sales recorded the lowest. This is due to the fact that the December month usually observe more number of sales because of the year end and festive season phase. For the positive side, company should maintain its production on the basis of median values which is around to 2000. By doing so,

company can fulfill the demands of the customers during the peak time and that also without any additional cost and any extra workers in the factory. If we are able to take these above steps carefully then definitely our sales of the Sparkling wine will improve in upcoming future.

_THE END__